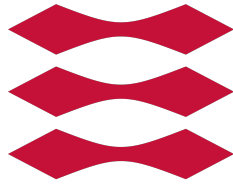


DTU



Danmarks Tekniske Universitet

A condensed summary of Introduction to Numerical Algorithms 02601

Author: Afonso Brito

Contents

1	Introduction	4
2	Mathematical Preliminaries	5
2.1	Notations	5
2.2	Horner's algorithm	5
2.3	Absolute and Relative errors	6
2.4	Rounding and chopping errors	6
2.5	Floating-Point Representation	6
2.5.1	Computer errors in representing numbers	6
2.6	Loss of significance	7
2.6.1	Tricks to avoid loss of significance	7
3	Data Fitting	8
3.1	Best fit function	8
3.2	Generalization of data fitting problems	8
3.2.1	Matrix vector notation	9
3.2.2	Normal equations	9
3.2.3	Basis functions	10
3.2.4	Computing best fit in Python	10
4	Methods for calculating the roots	11
4.1	Root	11
4.1.1	Multiple root	11
4.2	Comparison of methods	11
4.3	Bisection algorithm	12
4.3.1	Convergence analysis	12
4.3.2	Number of iterations for a given tolerance	13
4.3.3	Code	13
4.4	Newton's method	13
4.4.1	Convergence analysis	14
4.4.2	Code	15
4.5	Secant method	15
4.5.1	Convergence of secant method	16
4.6	Python's functions for computing roots	16
5	Polynomial interpolation	17
5.1	Basics	17
5.2	Interpolating polynomial: Lagrange form	17
5.3	Interpolating polynomial: Newton algorithm	18
5.4	Existence and uniqueness of polynomial interpolation	19
5.4.1	Nested multiplication	20
5.4.2	Flops for evaluating interpolation polynomial	20
5.5	Vandermonde matrix	20
5.6	Inverse interpolation	20
5.7	Errors in polynomial interpolation	20

5.7.1	First Interpolation error Theorem	21
5.7.2	Second Interpolation error Theorem	22
5.8	Mathematical overview	23
6	Numerical integration	24
6.1	Approximation through Interpolating polynomials	25
6.2	Newton-Cotes formulas	25
6.3	Trapezoid rule	25
6.3.1	Error analysis for trapezoid rule	26
6.3.2	Error analysis for trapezoid rule by Taylor series	26
6.4	Composite trapezoid rule	27
6.5	Simpson's rule	28
6.5.1	Error of Simpson's rule	29
6.6	Composite Simpson's rule	29
6.7	When the Polynomial-based formula is effective and accurate	30
6.7.1	Recursive adaptive integration	30
6.8	Gaussian Quadrature Theorem	31
6.9	Numerical integration with Scipy	32
7	Numerical Solution of Differential Equations: Initial-Value Problems	33
7.1	From Higher-Order Equations to First-Order Systems	33
7.2	Initial-Value Problems and Uniqueness	33
7.3	Euler's Method	34
7.4	Taylor Series Methods	35
7.5	Heun's and Midpoint Methods (Runge–Kutta 2)	35
7.6	Runge–Kutta 4th Order (RK4)	36
7.7	Error Comparison and Convergence	37
7.8	Adaptive Methods	38
7.9	Stability Analysis	38
8	Numerical Solutions of Systems and Boundary-Value Problems	40
8.1	Conceptual Difference Between IVPs and BVPs	40
8.2	Systems of First-Order Differential Equations	40
8.3	Euler's Method for Systems	40
8.4	Taylor Series Methods for Systems	41
8.5	Fourth-Order Runge–Kutta for Systems	41
8.6	Converting Higher-Order Equations to First-Order Systems	42
8.7	Boundary-Value Problems (BVPs)	42
8.8	Example BVP	42
8.9	The Shooting Method	42
8.9.1	Interpretation of the Shooting Parameter	43
8.9.2	Interpretation of the Shooting Function	43
8.9.3	Why Root-Finding Is Required	43
8.9.4	Role of the Step Size in the Shooting Method	43
8.9.5	Choosing the Step Size	44
8.9.6	Sanity Check	45
8.10	Interpolation of Discrete Values	45
8.11	Practical Implementation of a Linear BVP	45
8.12	Projectile with Quadratic Drag	45
8.12.1	Governing Equations	45
8.12.2	Simulation Setup	46
8.12.3	Finding the Hit Point ($y = 0$)	46
8.12.4	Mesh Refinement Error Estimate	46
8.12.5	Hitting a Target at (1000, 0)	46
8.12.6	Average Height over First 10 s	47

8.13	Summary	47
9	Linear system	48
9.1	Efficiency	48
9.2	Back substitution for upper triangular systems	49
9.2.1	Algorithm	49
9.2.2	Computational complexity	49
9.3	Gaussian elimination without pivoting	50
9.3.1	Example	50
9.3.2	Algorithm	51
9.4	LU-factorization	51
9.4.1	LU-factorization (no pivoting)	51
9.4.2	LU-factorization with pivoting	51
9.4.3	Solving a linear system using the LU-factorization	52
9.5	Pivoting	53
9.5.1	Mathematical reason	53
9.5.2	Numerical reason	53
9.5.3	Risk of loss of significance without pivoting	53
9.6	Gaussian elimination with pivoting	55
9.6.1	Gaussian elimination with partial pivoting	55
9.6.2	Gaussian elimination with scaled partial pivoting	55
9.6.3	Computational complexity of "Naive" Gaussian elimination vs Efficient implementation	56
10	Nonlinear equations	57
10.1	Newton's method for a single equation	57
10.2	Newton's method for a system of nonlinear equations	57
10.2.1	Linearization via Taylor expansion	58
10.2.2	Matrix form and Jacobian	58
10.3	Newton's iteration for nonlinear systems	59
10.4	Example: Newton's method for a 2×2 nonlinear system	59
11	Factorizations and Sensitivity	61
11.1	Sensitivity of a Simple Root	61
11.1.1	Error Bound for a Simple Root	61
11.1.2	Interpretation	61
11.1.3	Numerical Interpretation (Exam-Oriented)	62
11.2	LDL^T Factorization	62
11.2.1	Remarks on Applicability	62
11.3	Cholesky Factorization	63
11.3.1	Existence and Failure Cases	63
11.3.2	Example: 2×2 SPD Case	64
11.3.3	Solving a System via Cholesky	64
11.4	Vector and Matrix Norms	64
11.4.1	Vector 2-Norm	64
11.4.2	Matrix 2-Norm	65
11.4.3	Geometric Interpretation	65
11.5	Sensitivity of Linear Systems	65
11.5.1	Interpretation	66
11.5.2	Relative Error and Conditioning	66
11.5.3	Practical Meaning of the Condition Number	66
11.6	Residual and Backward Error	67
11.6.1	Forward vs Backward Error	67

Chapter 1

Introduction

Numerical algorithms are techniques for working with mathematical models, allowing problems to be solved using arithmetic and logical operations or on a computer, while ensuring that energy consumption and running time are desirable.

Therefore in order to use computers to solve problems, we need two tools, Mathematical models, to set up the problems, and numerical algorithms to solve the problems (efficiently and accurately).

Nowadays, engineering tasks are very complicated. They often involve several disciplines (mechanics, chemistry, physics, electronics, informatics, etc.). Most of these tasks can not be "solved by hand". Laboratory work is difficult and expensive (and not everything can be measured). lastly mathematical models and numerical algorithms make it possible to deal with larger and more complicated engineering tasks.

Course content

- Mathematical preliminaries (Taylor series, floating-point representation, loss of significance, etc.)
- Data fitting
- Methods for finding the roots
- Polynomial interpolation
- Numerical integration
- Numerical methods for solving differential equations (Initial values problems, boundary-value problems, etc.)
- Methods for solving linear systems
- Methods for solving nonlinear systems

Chapter 2

Mathematical Preliminaries

2.1 Notations

Sum,

$$\sum_{k=n}^m x_k = x_n + x_{n+1} + \cdots + x_{m-1} + x_m$$

Product,

$$\prod_{k=n}^m x_k = x_n \cdot x_{n+1} \cdots x_{m-1} \cdot x_m$$

Polynomial,

$$p(x) = \sum_{i=0}^n a_i x_i = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

Taylor's Theorem, For a function f with continuous derivatives of orders $0, 1, 2, \dots, (n+1)$ in $[a, b]$, then for any c and x in $[a, b]$:

$$f(x) = \sum_{k=0}^n \frac{f^{(k)}(c)}{k!} (x-c)^k + E_{n+1}$$

Where $f^{(k)}$ is the k th derivative of f .

$$E_{n+1} = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x-c)^{n+1}$$

Where ξ lies between c and x .

2.2 Horner's algorithm

Following mathematical expressions is not always the most efficient way to do the calculation. For example, to evaluate the polynomial:

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

requires n additions and $\frac{1}{2}n(n+1)$ multiplications.

$$p(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + xa_n) \cdots))$$

In this form, however, it only requires n additions and only n multiplications.

```
def HornersMethod(a, x): #a is an array of coefficients, x is the point where we want to evalua
    n = len(a)-1
    result = 0
    for i in range(n,-1,-1):
        result = result * x + a[i]
    return result
```

2.3 Absolute and Relative errors

When we say that the absolute error of $\bar{a}$ to a is ± 0.01 , it means

$$a - 0.01 \leq \bar{a} \leq a + 0.01, \text{ where } \bar{a} \text{ is an approximation}$$

When we say that the relative error to a is 0.01 (or 1%), it means

$$\frac{|a - \bar{a}|}{|a|} \leq 0.01$$

For practical reasons, the relative error is more meaningful and commonly used.

2.4 Rounding and chopping errors

Calculated results are affected by errors in data. How the error is propagated from the data to the results depends on the problem that we solve (equation, interpolation, etc.). But there are also other sources which introduce errors, like the computer, which is not a continuum, and can only accept a finite format. Computers will therefore round numbers by reducing the number of significant digits in one digit or chop discard all digits that follow the n th digit, but none of the remaining n digits are changed.

2.5 Floating-Point Representation

In the decimal system, a real number x can be represented as,

$$x = \pm \overbrace{d_0, d_1 d_2 d_3 \cdots}^{\text{mantissa}} \times 10^n, d_i = \text{decimal digits } (0, 1, 2, \dots, 9), n = \text{exponent}$$

In the binary system, x can be written as,

$$x = \pm 1, b_1 b_2 \dots b_{52} \times 2^k, b_i = \text{binary digits } (0, 1), k = \text{exponent}$$

Note that computers can only work with finite numbers.

The standard IEEE-754 has a word length of 64 bits,

- 1 bit for the sign $\pm$
- 52 bits for the mantissa $b_1 b_2 \dots b_{52}$
- 11 bits for the exponent, i.e., $-1022 \leq k \leq 1023$
- Special values of k is reserved for $\pm\text{inf}$ and NaN.

2.5.1 Computer errors in representing numbers

Since there are only 52 digits in the mantissa, errors can occur when we attempt to represent a real number x .

- We have to use the closest machine number, $fl(x)$, replacing x .
- It would leave to the roundoff error, $x - fl(x)$.
- It is calculated by rounding to obtain the last binary digit b_{52}

The relative error of this representation is bounded by,

$$\left| \frac{x - fl(x)}{x} \right| \leq 2^{-53} \approx 1.1102 \times 10^{-16}$$

Where $u = 2^{-53}$ is a unit roundoff error.

`np.finfo(float).eps` $\approx 2.2204 \times 10^{-16}$ in Python is the difference between 1 and the next larger number that can be stored in this machine, and we have $\text{eps} = 2u$. It is usually called as machine epsilon or machine precision.

Note that $fl(1 + u) = 1$ while $fl(1 + \text{eps}) \neq 1$.

2.6 Loss of significance

Assuming that a calculator works with floating-point numbers that have 10 decimal digits in the decimal system. set $x = \frac{1}{15}$ and calculating $z = x - \sin(x)$ in the calculator,

- input: $x \leftarrow 0.6666666667 \times 10^{-1}$
- Calculate: $y = \sin(x) \leftarrow 0.6661729492 \times 10^{-1}$
- Calculate: $x - y = 0.0004937175 \times 10^{-1}$
- Normalize: $z \leftarrow 0.4937175000 \times 10^{-4}$

As we can see the last three zeroes in z that are supplied by the calculator and are not correct, they are spurious zeroes.

This example shows one of the most common reasons for loss of significance, i.e., the subtraction of one quantity from another nearly equal quantity.

2.6.1 Tricks to avoid loss of significance

As an example the following function $f(x) = \sqrt{x^2 + 1} - 1$ with $x \ll 1$, two tricks, we define $f(x)$ as,

$$f(x) = \frac{(\sqrt{x^2 + 1} - 1)(\sqrt{x^2 + 1} + 1)}{\sqrt{x^2 + 1} + 1} = \frac{x^2}{\sqrt{x^2 + 1} + 1}$$

or as (using Taylor-expansion),

$$f(x) = \frac{1}{2}x^2 - \frac{1}{8}x^4 + O(x^6)$$

If the quadratic equation $ax^2 + bx + c = 0$ has roots, they are given by the quadratic formula,

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad \vee \quad x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

Potential loss of significance in subtraction if $\sqrt{b^2 - 4ac} \approx |b|$. We then compute the root that avoids the subtraction first. Then, obtain the other root based on $x_1 x_2 = c/a$.

Chapter 3

Data Fitting

If we are given data $(x_k, y_k), k = 0, 1, 2, \dots, m$ with measurement errors and we want to fit a model a function F to these data. The model captures the "overall behavior" of the data without being too sensitive to the errors. We usually assume that the abscissas x_k appear in non-decreasing order, i.e., $x_0 \leq x_1 \leq \dots \leq x_m$. We assume that we have $m + 1$ data points, which can be approximately described by the function F , i.e., $y_k \approx F(x_k), k = 0, 1, 2, \dots, m$.

We introduce the error associated with the data points and the fit function F as,

$$e_k = y_k - F(x_k), k = 0, 1, 2, \dots, m$$

The absolute error $|e_k|$ is the vertical distance between the fit function and the data point. We want to find the "best" fit function F to the data, which requires that the function F has a suitable number of degrees of freedom.

3.1 Best fit function

To find the best fit function, we can minimize the sum of the absolute errors,

$$\min \sum_{k=0}^m |e_k| = \min \sum_{k=0}^m |f(x_k) - y_k|$$

In theory, this is good, but the problem is difficult and expensive to solve (linear programming). Therefore, we minimize the sum of squares of the errors,

$$\min \sum_{k=0}^m e_k^2 = \min \sum_{k=0}^m (F(x_k) - y_k)^2$$

This is easy to solve, and if the errors follow a normal probability distribution, then the minimization produces a best estimate of F .

3.2 Generalization of data fitting problems

Supposing that the fit function F is a linear combination of a set known basis functions,

$$F(x) = \sum_{j=0}^n c_j g_j(x)$$

The coefficients c_j are to be determined to provide the least squares fit. The function F should minimize,

$$\varphi(c_0, c_1, \dots, c_n) = \sum_{k=0}^m (F(x_k) - y_k)^2 = \sum_{k=0}^m \left(\sum_{j=0}^n c_j g_j(x_k) - y_k \right)^2$$

The necessary conditions for the minimum are,

$$\frac{\partial \varphi}{\partial c_j} = 0, \text{ for } j = 0, 1, \dots, n$$

3.2.1 Matrix vector notation

We defined,

$$\mathbf{A} = \begin{bmatrix} g_0(x_0) & g_1(x_0) & \cdots & g_n(x_0) \\ g_0(x_1) & g_1(x_1) & \cdots & g_n(x_1) \\ g_0(x_2) & g_1(x_2) & \cdots & g_n(x_2) \\ g_0(x_3) & g_1(x_3) & \cdots & g_n(x_3) \\ \vdots & \vdots & & \vdots \\ g_0(x_n) & g_1(x_n) & \cdots & g_n(x_n) \end{bmatrix}, \mathbf{c} = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}, \mathbf{y} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_m \end{bmatrix}$$

Where $\mathbf{A}$ is an $(m+1) \times (n+1)$ matrix, $\mathbf{c}$ is a $n+1$ vector, and $\mathbf{y}$ is an $m+1$ vector. The error vector, which depends on $\mathbf{c}$,

$$\mathbf{r}(\mathbf{c}) = \begin{bmatrix} \sum_{j=0}^n c_j g_j(x_0) - y_0 \\ \sum_{j=0}^n c_j g_j(x_1) - y_1 \\ \vdots \\ \sum_{j=0}^n c_j g_j(x_m) - y_m \end{bmatrix} = \mathbf{A}\mathbf{c} - \mathbf{y}$$

The least squares fit function F should minimize,

$$\varphi(c_0, c_1, \dots, c_n) = \sum_{k=0}^m \mathbf{r}(\mathbf{c})_{k+1}^2$$

Which shows the distance between $\mathbf{A}\mathbf{c}$ and $\mathbf{y}$.

3.2.2 Normal equations

When $\mathbf{r}(\mathbf{c}) \perp \mathbf{A}\mathbf{c}$, we obtain the shortest $\mathbf{r}(\mathbf{c}) = \mathbf{A}\mathbf{c} - \mathbf{y}$. That is, $\mathbf{r}(\mathbf{c}) = \mathbf{A}\mathbf{c} - \mathbf{y}$ must be orthogonal to all columns of $\mathbf{A}$,

$$\mathbf{A}^\top \mathbf{r}(\mathbf{c}) = 0 \Leftrightarrow \mathbf{A}^\top \mathbf{A}\mathbf{c} - \mathbf{A}^\top \mathbf{y} = 0 \Leftrightarrow \mathbf{A}^\top \mathbf{A}\mathbf{c} = \mathbf{A}^\top \mathbf{y}$$

Which is the normal equations.

Log-linear least-squares fit. To study error decay of the form

$$e_n \approx \beta n^{-\alpha},$$

we take logarithms and obtain the linear model

$$\log(e_n) \approx \log(|\beta|) - \alpha \log(n) = b + m \log(n),$$

with unknowns $b = \log(|\beta|)$ and $m = -\alpha$.

Let

$$x_i = \log(n_i), \quad y_i = \log(e_{n_i}),$$

for $n_i \in \{20, 40, 60, 80, 100\}$. The least-squares problem can then be written as

$$Au \approx y, \quad A = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ 1 & x_4 \\ 1 & x_5 \end{pmatrix}, \quad u = \begin{pmatrix} b \\ m \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{pmatrix}.$$

The corresponding normal equation is

$$A^\top Au = A^\top y,$$

that is,

$$\begin{pmatrix} 5 & \sum_{i=1}^5 x_i \\ \sum_{i=1}^5 x_i & \sum_{i=1}^5 x_i^2 \end{pmatrix} \begin{pmatrix} b \\ m \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^5 y_i \\ \sum_{i=1}^5 x_i y_i \end{pmatrix}.$$

Finally, the decay rate is recovered as $\alpha = -m$.

3.2.3 Basis functions

Requirements for the basis functions g_j .

- The behavior of the fit function F strongly depends on the basis functions.
- They should be appropriate to describe the data.
 - Monotonically decreasing data $\rightarrow$ exponential.
 - Periodic data $\rightarrow$ sine and cosine
- They must be linearly independent, i.e.,

$$\alpha_0 g_0(x) + \alpha_1 g_1(x) + \cdots + \alpha_n g_n(x) = 0$$

if and only if $\alpha_0 = \alpha_1 = \cdots = \alpha_n = 0$.

- Because of linear independence, the matrix $\mathbf{A}^T \mathbf{A}$ in the normal equations has full rank, so the least-squares solution exists and is unique.
- If the condition of linear independence is not satisfied, the matrix $\mathbf{A}^T \mathbf{A}$ is singular, and we cannot solve the normal equations.

3.2.4 Computing best fit in Python

For given data and non-polynomial basis functions the code is as follows,

```
x = np.array([...]) #x data points
y = np.array([...]) #y data points
A = np.array([[...]] for xi in x) #basis functions evaluated at x data points
A.T @ A, A.T @ y
```

To compute the c_j constants we simply run the the following code after the previous one,

```
x = np.array([])
y = np.array([])
c0, c1 = symbols('c0 c1')
cons = Matrix([c0, c1])
x_sy = Matrix(x)
y_sy = Matrix(y)
M = Matrix([[...]] for xi in x_sy)
c = solve((M.T*M)*cons - M.T*y_sy, (c0,c1))
c
```

For given data whose basis function is an n th-order polynomial, the following code is the most appropriate,

```
import numpy as np
x = np.array([...])
y = np.array([...])
n = #whatever order you desire
z = np.polyfit(x, y, n)
#will return the values of the constants a0, a1, a2, ..., an

p = np.poly1d(z) #this will give you the polynomial as a function making you able to plot the f

xp = np.linspace(min, max, 100)
_ = plt.plot(x, y, '.', xp, p(xp), '-', xp, p(xp), '--')
plt.show()
```

Chapter 4

Methods for calculating the roots

This chapter lists three methods to calculate a root of an equation, especially a nonlinear equation. If we can find a root of an equation then we also can calculate the minimum for a function, since a function f reaches to its minimum as $f'(x) = 0$.

4.1 Root

A number r , which satisfies $f(r) = 0$, is called as a root of that equation, or a zero (or root) of that function.

In the most cases we cannot find a root r analytically, then we need numerical methods to get a good approximation of the root,

Suppose that $f(x)$ is continuous. Let,

$$a < b \text{ and } f(a) \cdot f(b) < 0$$

i.e. $f(a)$ and $f(b)$ have opposite signs. it then follows that f has a root in the interval $[a, b]$.

4.1.1 Multiple root

- Simple root: $f(r) = 0$ and $f'(r) \neq 0$
- Double root: $f(r) = f'(r) = 0$ and $f''(r) \neq 0$
- Triple root: $f(r) = f'(r) = f''(r) = 0$ and $f'''(r) \neq 0$

4.2 Comparison of methods

- **Bisection** is robust but slow. If an error tolerance has been given in advance, the number of iterations required by the bisection method can be estimated.
- **Bisection** requires two starting points with different signs of $f(x)$, and f only needs to be continuous.
- **Newton's method** requires f' .
- **Newton's method** and **secant method** converge faster than the bisection method, but require a starting point that is "sufficiently close" to a root.
- **Secant method** is nearly as fast as Newton's method and does not require knowledge of the derivative f' .
- **All methods** require the user to know roughly about the desired root and how many roots to be examined.

Method	Requirements	Convergence Order	Pros / Cons
Bisection	Sign change on $[a, b]$	Linear	Very robust; slow
Newton	Derivative f' needed	Quadratic (if close)	Fast; may diverge if x_0 is poor
Secant	Two starting values	≈ 1.62 (superlinear)	No derivative needed; less stable than Newton

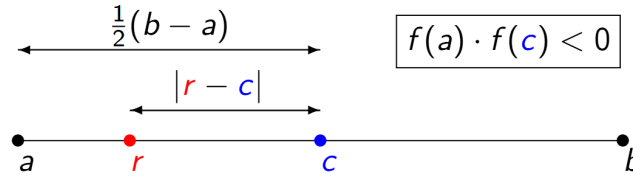
Table 4.1: Comparison of the main root-finding methods.

4.3 Bisection algorithm

Given two values a and b , where $a < b$ and $f(a)$ and $f(b)$ have opposite signs.

We calculate the midpoint $c = (a + b)/2$ of the interval $[a, b]$. If $f(a) \cdot f(c) < 0$, then we know that there is a root of f in the interval $[a, c]$, and the absolute error is bounded as,

$$|r - c| \leq \frac{1}{2}(b - a)$$



In the same way, as the interval becomes smaller and smaller, we approach to a root of f . Absolute error after n iterations is computed using the following formula,

$$|r - c| \leq \frac{b - a}{2^{n+1}}$$

The bisection algorithm

- Is robust, but converges slowly.
- It cannot find some multiple roots.
- If f is not continuous, we cannot use the bisection algorithm.
- It cannot be generalized to the case with more than one variable.

4.3.1 Convergence analysis

If we use a midpoint $c_0 = (a_0 + b_0)/2$ as our estimate of r , we have,

$$|r - c_0| \leq \frac{b_0 - a_0}{2}$$

Then after n iterations, the maximum error will be reduced by a factor of 2^n :

$$|r - c_n| \leq \frac{b_n - a_n}{2} = \frac{\frac{1}{2}(b_{n-1} - a_{n-1})}{2} \leq \frac{b_0 - a_0}{2^{n+1}} = \frac{1}{2^n} \frac{b_0 - a_0}{2}$$

We won't have linear convergence, since there is no constant $C \in [0, 1[$, such that

$$\forall n \geq 1 : |c_{n+1} - r| \leq C|c_n - r|$$

Since there is a possibility that the error increases from one iteration to the next (i.e. $C > 1$).

4.3.2 Number of iterations for a given tolerance

The number of iterations, n , of the bisection algorithm are needed to guarantee that the error is bounded by a given tolerance ϵ , can be calculated by solving the inequality,

$$|r - c_n| \leq \frac{b - a}{2^{n+1}} < \epsilon$$

We then obtain,

$$n = \text{ceil}\left(\frac{\ln(b - a) - \ln(2\epsilon)}{\ln 2}\right)$$

4.3.3 Code

Code to find the guess after n iterations of bisection method

```
def bisection_method(f, a, b, nmax):
    fa = f(a)
    fb = f(b)
    for _ in range(nmax):
        c = (a + b) / 2
        fc = f(c)
        if abs(fc) < np.finfo(float).eps:
            return c
        if fa * fc < 0:
            b = c
            fb = fc
        elif fb * fc < 0:
            a = c
            fa = fc
    return (a + b) / 2
```

Code to find the root of a function using the bisection method (will run indefinitely)

```
def bisection_method_indef(f, a, b):
    fa = f(a)
    fb = f(b)
    while (b - a) / 2 > np.finfo(float).eps:
        c = (a + b) / 2
        fc = f(c)
        if abs(fc) < np.finfo(float).eps:
            return c
        if fa * fc < 0:
            b = c
            fb = fc
        elif fb * fc < 0:
            a = c
            fa = fc
    return (a + b) / 2
```

Code to find the specific number of iterations n for a given tolerance,

```
def bisection_n_iter_midpoint(r, a, b):
    return int(np.ceil(np.log2((b - a) / r)) - 1)
```

4.4 Newton's method

Based on Taylor's Theorem at $x = x_0$ we have,

$$f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{1}{2}h^2 f''(x_0)$$

We want to find h such that $f(x_0 + h) = 0$, but it's not easy. If we ignore all higher-order terms in the Taylor series and find h satisfying,

$$f(x_0) + hf'(x_0) = 0$$

Which gives,

$$h = -\frac{f(x_0)}{f'(x_0)}$$

Then algorithm: x_0 = starting point, for $n = 0, 1, 2, \dots$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

$\varsigma = |x_{n+1} - x_n|$ is an estimate for the error $|r - x_n|$, so stop as $\varsigma_n < \epsilon$.

Newton's method is not robust. When the starting point is sufficiently close to a root, Newton's method converges fast. But if x_0 is far from r

4.4.1 Convergence analysis

The formula of x_{n+1} from Newton's method gives,

$$e_{n+1} \equiv r - x_{n+1} = \underbrace{r - x_n}_{e_n} + \frac{f(x_n)}{f'(x_n)} = \frac{e_n f'(x_n) + f(x_n)}{f'(x_n)}$$

By Taylor's Theorem, there exists a point ξ_n situated between r and x_n for which,

$$\begin{aligned} 0 = f(r) = f(x_n + e_n) &= f(x_n) + e_n f'(x_n) + \frac{1}{2} e_n^2 f''(\xi_n) \Leftrightarrow \\ \Leftrightarrow e_n f'(x_n) + f(x_n) &= -\frac{1}{2} e_n^2 f''(\xi_n) \end{aligned}$$

Substituting in e_{n+1}

$$e_{n+1} = \frac{-\frac{1}{2} e_n^2 f''(\xi_n)}{f'(x_n)} = -\frac{1}{2} \left(\frac{f''(\xi_n)}{f'(x_n)} \right) e_n^2$$

This is, at least quantitatively, the sort of equation we want.

Suppose that at some iterate x_n we have $|e_n| = |r - x_n| \leq \delta \Rightarrow |r - \xi_n| \leq \delta$.

Define a function,

$$c(\delta) \equiv \frac{1}{2} \frac{\max_{|r-x| \leq \delta} |f''(x)|}{\min_{|r-x| \leq \delta} |f'(x)|}$$

Then we have,

$$|e_{n+1}| = \frac{1}{2} \left| \frac{f''(\xi_n)}{f'(x_n)} \right| e_n^2 \leq c(\delta) e_n^2 \leq c(\delta) |e_n| \delta = \delta c(\delta) |e_n| \quad (4.1)$$

In the same way, if $|e_{n-1}| = |r - x_{n-1}| \leq \delta$, then we have,

$$|e_n| \leq \delta c(\delta) |e_{n-1}|$$

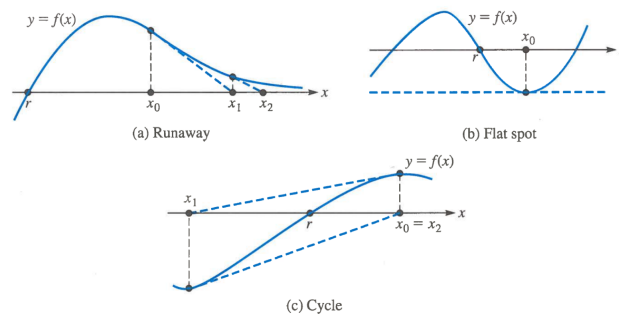
Continue backward until the first iteration, and obtain that if $|r - x_0| \leq \delta$ we have,

$$|e_{n+1}| \leq (\delta c(\delta))^{n+1} |e_0|$$

Hence, Newton's method always converges if x_0 is close enough to r , i.e., $\delta c(\delta) < 1$. Its convergent rate is quadratic according to 4.1.

Important to note that $|e_n/e_{n-1}^2|$ tends to a constant when the correct multiplicity m is applied in the code.

The convergence of Newton's method depends on the hypothesis that the starting point x_0 is sufficiently close to a root. If x_0 is a bit far away from r , the method can fail.



4.4.2 Code

Code for Newton's method guess after n iterations,

```
def Newtons_method(f,df,x0,m,nmax):
    #m is the multiplicity of the root
    xx = []
    fxx = []
    while len(xx) <= nmax:
        xx.append(x0)
        fxx.append(f(x0))
        x0 = x0 - m*f(x0)/df(x0)
    return xx
```

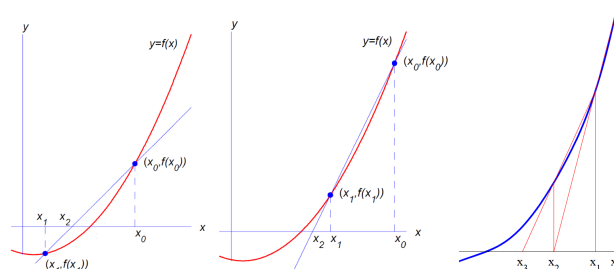
Code for a table and checking quadratic convergence,

```
def Newtons_method_w_table_updated(f,df,x0,r,m,nmax):
    #r is the root
    #m is multiplicity of the root
    T = np.zeros((nmax+1,4))
    c = 0
    while c <= nmax:
        T[c,0] = c
        T[c,1] = x0
        T[c,2] = abs(r - x0)
        T[c,3] = abs(T[c,2]/T[c-1,2]**2) if c>0 else np.nan
        x0 = x0 - m*f(x0)/df(x0)
        c += 1
    return T
```

4.5 Secant method

In some cases it's difficult to obtain $f'(x_n)$, for example, it's too expensive to compute $f'(x_n)$ or we lack knowledge of the function and its derivatives, then Newton's method cannot be used.

Alternatively: We approximate $f'(x_n)$ by the slope of the secant line of two points x_{n-1} and x_n .



The equation of the secant line from $(x_{n-1}, f(x_{n-1}))$ and $(x_n, f(x_n))$ is,

$$y = f(x_n) + \frac{f(x_{n-1}) - f(x_n)}{x_{n-1} - x_n}$$

If we set $y = 0$, then we obtain

$$x = x_{n+1} = x_n - f(x_n) \frac{x_{n-1} - x_n}{f(x_{n-1}) - f(x_n)}$$

As an approximation of Newton's method we use an approximation of $f'(x_n)$,

$$f'(x_n) \approx \frac{f(x_{n-1}) - f(x_n)}{x_{n-1} - x_n}$$

Newton's iteration can be approximated by,

$$x_{n+1} = x_n - f(x_n) \frac{x_{n-1} - x_n}{f(x_{n-1}) - f(x_n)}$$

4.5.1 Convergence of secant method

The order of convergence of the secant method can be expressed in terms of the inequality,

$$|e_{n+1}| \leq C|e_n|^\phi, \quad C < 1 \text{ (a constant)}$$

where $\phi = \frac{1}{2}(1 + \sqrt{5}) \approx 1.62$ is the golden ratio. Since $\phi > 1$, we say that the method has superlinear convergence, which is faster than linear.

Code for Secant's method guess after n iterations,

```
def secant_min(f, x0, x1, nmax):
    xx = []
    xx.append(x0)
    xx.append(x1)
    fa=f(xx[0])
    fb=f(xx[1])
    for n in range(2, nmax+1):
        xnext = xx[n-1] - fb*(xx[n-1]-xx[n-2])/(fb-fa)
        xx.append(xnext)
        fa=fb
        fb=f(xnext)
    return xx
```

4.6 Python's functions for computing roots

The Python package Scipy has the following functions that calculate roots,

```
#Import root-finding functions
from scipy.optimize import fsolve, brent

#finds a root of the function f near x0
x = fsolve(f, x0)

#finds a root of the scalar function f
x = root_scalar(f)

#finds a root in the interval [a,b], the signs of f(a) and f(b) must be different
x = brentq(f,a,b)
```

Chapter 5

Polynomial interpolation

Finding a function like a polynomial or other types of functions to represent data can help in the following instances,

- When we have a finite number of data points and if we can find a simple formula to reproduce the given points exactly, then we can estimate the value y for any x .
- If the data is corrupted with errors, then we would looking for a formula that represents the data and filters out the errors
- A function p is given but (energetically) expensive to evaluate. Then we can find another simpler function to approximate p , which is easier to be calculated, integrated and/or differentiated

5.1 Basics

Considering a table of values,

x	x_0	x_1	$\cdots$	x_n
y	y_0	y_1	$\cdots$	y_n

Table 5.1

We assume that the x_i 's form a set of $n + 1$ distinct points, called nodes, i.e., $x_i \neq x_j$ if $i \neq j$. A function p interpolates data $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, if,

$$p(x_i) = y_i \text{ for } i = 0, 1, \dots, n$$

5.2 Interpolating polynomial: Lagrange form

The Lagrange polynomial that interpolates $n + 1$ points will be at most an n th degree polynomial. To interpolate a table of values 5.1 we first define cardinal polynomials l_i for $i = 0, \dots, n$ which are defined as,

$$l_i(x) = \prod_{j=0, j \neq i}^n \lambda_{i,j}(x) = \prod_{j=0, j \neq i}^n \left(\frac{x - x_j}{x_i - x_j} \right)$$

Code for computing the i th cardinal polynomial is,

```
def CardinalPolynomial(nodes,i,t):  
    # Computes the i-th cardinal polynomial at t  
    L = 1  
    for j in range(len(nodes)):  
        if j != i:  
            L *= (t - nodes[j])/(nodes[i] - nodes[j])  
    return L
```

Where $\lambda_{i,j}$ satisfies $\lambda_{i,j}(x_i) = 1$ and $\lambda_{i,j}(x_j) = 0$. Hence, the cardinal polynomial l_i has the property,

$$l_i(x_j) = \delta_{ij} = \begin{cases} 1, & \text{if } i = j \\ 0, & \text{if } i \neq j \end{cases}$$

Therefore, to interpolate a table of values 5.1, the Lagrange form of the interpolation polynomial is,

$$p_n(x) = \sum_{i=0}^n l_i(x) y_i$$

and for each x_j ,

$$p_n(x_j) = \sum_{i=0}^n l_i(x_j) y_i = 0 + \cdots + 0 + y_j + 0 + \cdots + 0 = y_j$$

We can approximate any function, f , by interpolating a table of its values.

It is important to note that cardinal polynomials only depend on the values of the nodes (x -values), and not y -values. The function values (y -values) are only used as weights for the cardinal polynomials in order to obtain the Lagrange form,

$$p_n(x) = \sum_{i=0}^n l_i(x) f(x_i) \Leftrightarrow p_n(x) = \sum_{i=0}^n l_i(x) y_i$$

We would use the first formula when trying to find a simpler function to approximate a more energetically expensive function. Code for computing the Lagrange Interpolation polynomial at t is,

```
def InterpolerLagrangeForm(nodes, values, t):
    # Computes the Lagrange interpolation polynomial at t
    P = 0
    for i in range(len(nodes)):
        P += values[i] * CardinalPolynomial(nodes, i, t)
    return P
```

5.3 Interpolating polynomial: Newton algorithm

The Newton algorithm constructs a sequence of polynomials, each of which interpolates one more pair of values in the table than the predecessor.

x	x_0	x_1	$\cdots$	x_k	x_{k+1}	$\cdots$	x_n
y	y_0	y_1	$\cdots$	y_k	y_{k+1}	$\cdots$	y_n

Table 5.2

Assuming that the polynomial p_k is of degree at most k and interpolates the first $k + 1 < n$ pairs of values in the table, i.e., $p_k(x_i) = y_i$ for $i = 0, \dots, k$.

now we consider,

$$P(x) = p_k(x) + c(x - x_0)(x - x_1) \cdots (x - x_{k-1})$$

$P(x)$ is of degree at most $(k + 1)$, and the constant c can be uniquely determined by,

$$P(x_{k+1}) = p_k(x_{k+1}) + c(x_{k+1} - x_0)(x_{k+1} - x_1) \cdots (x_{k+1} - x_k) = y_{k+1}$$

Then, we have $p_{k+1} = P$

The final Newton's algorithm for interpolating polynomial will then look like,

$$P(x) = y_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n(x - x_0) \cdots (x - x_{n-1})$$

Where each c_i is a constant computed as,

$$c_i = \frac{y_i - p_{i-1}(x_i)}{(x_i - x_0) \cdots (x_i - x_{i-1})}$$

Doing it in Python as,

```

def InterloperNewtonForm(nodes, values, t):
    # Computes the Newton interpolation polynomial at t
    n = len(nodes)
    coeffs = np.copy(values).astype(float)
    coeffs[0] = values[0]

    for j in range(1, n):
        coeffs[j:n] = (coeffs[j:n] - coeffs[j-1:n-1]) / (nodes[j:n] - nodes[0:n-j])

    def P(x):
        """Evaluate polynomial at x (scalar or array)."""
        x = np.asarray(x)
        result = np.zeros_like(x, dtype=float)
        for idx, xi in np.ndenumerate(x):
            term = 1.0
            yi = coeffs[0]
            for j in range(1, n):
                term *= (xi - nodes[j-1])
                yi += coeffs[j] * term
            result[idx] = yi
        return result

    return P, coeffs

```

To get the polynomial and the coefficients c_i not just a value we can use,

```

nodes = np.array([1,-2,0])
values = np.array([3,-3,-7])
t = np.linspace(-3, 3, 100)
P, coeffs = InterloperNewtonForm(nodes, values, t)
print("Divided Differences Coefficients:", coeffs)
x = symbols('x')
P_sym = coeffs[0]
term = 1
for j in range(1, len(nodes)):
    term *= (x - nodes[j-1])
    P_sym += coeffs[j] * term
print("Newton Form Polynomial:", P_sym.expand())

```

5.4 Existence and uniqueness of polynomial interpolation

Theorem 5.4.1. *If points $x_0, x_1, \dots, x_n$ are distinct, then for arbitrary real values $y_0, y_1, \dots, y_n$ there is a unique polynomial p of degree at most n such that,*

$$p(x_i) = y_i \text{ for } i = 0, 1, \dots, n$$

Proof of existence (inductive reasoning): **Starting with $n = 0$:** The pair of values (x_0, y_0) can be uniquely interpolated by the 0-degree polynomial $p_0(x) = y_0$

Induction stage: Based on the Newton algorithm, we can uniquely determine a constant c , which leads to a polynomial of degree at most n .

Uniqueness: If both polynomials p and q interpolate the table, then the polynomial $p - q$ is of degree at most n and takes the value 0 at all $n + 1$ distinct points $x_0, x_1, \dots, x_n$. However, a nonzero polynomial can have at most n roots. Therefore, we conclude that $p = q$.

Therefore, the Newton form and Lagrange form are just two different derivations for precisely the same polynomial.

5.4.1 Nested multiplication

Based on the Newton algorithm we can rewrite the interpolating polynomial by "nested multiplication",

$$\begin{aligned}
p_n(x) &= c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}) \\
&= c_0 + (x - x_0)(c_1 + c_2(x - x_1) + c_3(x - x_1)(x - x_2) + \cdots + c_n(x - x_1)(x - x_2) \cdots (x - x_{n-1})) \\
&= c_0 + (x - x_0)(c_1 + (x - x_1)(c_2 + c_3(x - x_2) + \cdots + c_n(x - x_2) \cdots (x - x_{n-1}))) \\
&\vdots \\
&= c_0 + (x - x_0) \left(c_1 + (x - x_1) (c_2 + \cdots + (x - x_{n-2}) (c_{n-1} + (x - x_{n-1}) c_n)) \cdots \right)
\end{aligned} \tag{5.1}$$

5.4.2 Flops for evaluating interpolation polynomial

The Lagrange form consists of $n + 1$ cardinals of degree n .

$$l_i(x) = \prod_{j=0, j \neq i}^n \left(\frac{x - x_j}{x_i - x_j} \right)$$

To evaluate each cardinal polynomial, we need $2n$ subtraction, n divisions, and $n - 1$ multiplications. So to evaluate the whole Lagrange form we need $4n(n + 1) + n$ FLOPS (float point operations).

Newton algorithm provides interpolation polynomials in a nested multiplication, which need much less FLOPS for evaluation.

Since with nested multiplication it only requires $3n$ FLOPS.

5.5 Vandermonde matrix

Another way of finding a polynomial $p_n(x) = c_0 + c_1x + c_2x^2 + \cdots + c_nx^n$ to interpolate a table is to solve $n + 1$ equations, $p_n(x_i) = y_i$, with $n + 1$ unknown coefficients $c_0, c_1, \dots, c_n$. This can be written as a linear system,

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

The coefficient matrix is called a Vandermonde matrix. For $n \approx 10$ it can already have numerical issue to solve it.

5.6 Inverse interpolation

If we reverse the arguments x and $y = f(x)$ in the table for interpolation, then we would obtain an approximation, $x = p(y)$, of the inverse function $f^{-1}(y)$.

The inverse interpolation is often used to approximate an inverse function.

Inverse interpolation can be used to find a root of the given function. We create a table of values $(f(x_i), x_i)$ and interpolate with a polynomial, p . Thus, we obtain $p(y_i) = x_i$, and the approximate root is then given by $p(0)$.

Note: the points x_i should be chosen near the unknown root.

5.7 Errors in polynomial interpolation

The upper bound of the error for interpolation can be estimated by the first or second Interpolation error Theorem, explained below.

Approximation by interpolation doesn't always work well. It requires that "high-order derivative" of f has a small value.

5.7.1 First Interpolation error Theorem

A problem with high-degree polynomials is that a polynomial of degree n has n zeros, i.e., its curve crosses the x -axis n times, which results in wild oscillations.

We can assess when interpolation works well, using the following,

Theorem 5.7.1. First interpolation Error: *If p_n is the polynomial of degree at most n that interpolates f at the $n + 1$ distinct nodes $x_0, x_1, \dots, x_n \in [a, b]$ and if $f^{(n+1)}$ is continuous, then for each $x \in [a, b]$, there is a $\xi \in]a, b[$ for which,*

$$f(x) - p_n(x) = \frac{1}{(n+1)!} f^{(n+1)}(\xi) \prod_{i=0}^n (x - x_i)$$

Obviously, the equation is valid for all nodes. Now, we need to show that the equation is also valid for the case that x is not a node.

Defining, $\omega(t) = \prod_{i=0}^n (t - x_i)$, which equals to zero at all nodes $x_0, x_1, \dots, x_n$, and introduce a constant c as,

$$c = \frac{f(x) - p_n(x)}{\omega(x)}$$

Which is defined because $\omega(x) \neq 0$

Defining a function in the variable t :

$$\varphi(t) = f(t) - p_n(t) - c\omega(t)$$

Note that $f(t) - p_n(t)$ and $\omega(t)$ equal to zero at all nodes. Hence, $\varphi(t)$ also equals to zero at all nodes. Moreover, according to the definition of c , $\varphi(t)$ equals to 0 at $t = x$. That is,

- φ takes the value 0 at the $n + 2$ points $x_0, x_1, \dots, x_n$ and x .
- Rolle's Theorem states that $\varphi' = 0$ for at least $n + 1$ points between $x_0, x_1, \dots, x_n$ and x .
- Rolle's Theorem states that $\varphi'' = 0$ for at least n points.
- $\varphi''' = 0$ for at least $n - 1$ points.
- $\vdots$
- Rolle's Theorem states that $\varphi^{(n+1)} = 0$ for at least one point $\xi \in]a, b[$, that is

$$0 = \varphi^{(n+1)}(\xi) = f^{(n+1)}(\xi) - p_n^{(n+1)}(\xi) - c\omega^{(n+1)}(\xi)$$

In this equation, we know that,

$$\omega(t) = \prod_{i=0}^n (t - x_i) = t^{n+1} + (\text{lower order terms})$$

So,

$$\omega^{(n+1)}(t) = \omega^{(n+1)}(\xi) = (n+1)!$$

p_n is a polynomial of degree $\leq n$, so $p_n^{(n+1)}(\xi) = 0$

Thus we have,

$$\begin{aligned} 0 &= \varphi^{(n+1)}(\xi) = f^{(n+1)}(\xi) - 0 - c(n+1)! \\ &= f^{(n+1)}(\xi) - \frac{f(x) - p_n(x)}{\omega(x)}(n+1)! \end{aligned}$$

$\Updownarrow$

$$f(x) - p_n(x) = \frac{1}{(n+1)!} f^{(n+1)}(\xi) \prod_{i=0}^n (x - x_i)$$

Therefore, proving it for all x 's, even if they're not a node.

```

import numpy as np
import numdifftools as nd
import math

def first_interpolation_error_theorem(f, a, b, n):
    """
    Computes the interpolation error bound using
    the First Interpolation Error Theorem.

    Parameters
    -----
    f : callable
        Function to interpolate (e.g. np.sin)
    a, b : float
        Interval endpoints
    n : int
        Degree of interpolating polynomial (n = number of nodes - 1)
    """
    f_deriv = nd.Derivative(f, n+1) # (n+1)-th derivative
    xs = np.linspace(a, b, 400)
    M = np.max(np.abs(f_deriv(xs))) # max |f^(n+1)(x)|
    error_bound = (M / math.factorial(n+1)) * ((b - a) ** (n+1))
    return error_bound

```

5.7.2 Second Interpolation error Theorem

Often, the interpolation nodes are equally spaced in $[a, b]$, so $x_{i+1} - x_i = h = (b - a)/n$. Then we can obtain that for any $x \in [a, b]$,

$$\prod_{i=0}^n |x - x_i| \leq \frac{1}{4} h^{n+1} n!$$

Thus, we have,

$$\left| f(x) - p_n(x) \right| = \frac{1}{(n+1)!} \left| f^{(n+1)}(\xi) \right| \prod_{i=0}^n |x - x_i| \leq \frac{1}{4(n+1)} \left| f^{(n+1)}(\xi) \right| h^{n+1}$$

Moreover, if $\left| f^{(n+1)}(x) \right| \leq M$ for all $x \in [a, b]$, then we find a bound independent from x :

Theorem 5.7.2. Second Interpolation Error: Let f be a function such that $f^{(n+1)}$ is continuous on $[a, b]$ and satisfies $|f^{(n+1)}(x)| \leq M$. Let p_n be the polynomial of degree at most n that interpolates f at the $n + 1$ equally spaced nodes in $[a, b]$, including the endpoints. Then, for all $x \in [a, b]$, we have,

$$\left| f(x) - p_n(x) \right| \leq \frac{1}{4(n+1)} \left| f^{(n+1)}(\xi) \right| h^{n+1} \leq \frac{1}{4(n+1)} M h^{n+1}$$

Where $h = (b - a)/n$ is the spacing between the nodes.

Doing it in Python as,

```

def second_interpolation_error_theorem(f, a, b, n):
    """
    Error bound from Theorem 5.7.2
    Assumes n+1 equally spaced nodes including endpoints.
    """

```

```

import numpy as np
import numdifftools as nd

f_deriv = nd.Derivative(f, n = n + 1)
xs = np.linspace(a, b, 400)
M = np.max(np.abs(f_deriv(xs)))

h = (b - a) / n
return (M * h**(n + 1)) / (4 * (n + 1))

```

5.8 Mathematical overview

- Considering a fixed selected set of nodes x_i , for $i = 0, \dots, n$.
- $p_{1,n}$ approximates the function f_1 : $p_{1,n}(x_i) = f_1(x_i)$ for $i = 0, \dots, n$
- $p_{2,n}$ approximates the function f_2 : $p_{2,n}(x_i) = f_2(x_i)$ for $i = 0, \dots, n$
- Let $a, b \in \mathbb{R}$, The polynomial that approximates $af_1 + bf_2$ is $ap_{1,n} + bp_{2,n}$, because,

$$ap_{1,n}(x_i) + bp_{2,n}(x_i) = af_1(x_i) + bf_2(x_i) \text{ for } i = 0, \dots, n$$

And this polynomial is uniquely determined.

Hence a polynomial interpolation can be considered as a linear mapping from a function space to the space of polynomials.

Corollary 5.8.0.1. *The monomials x^i , $i = 0, \dots, n$ are linear independent functions.*

Assuming a linear combination of the monomials equals to 0, i.e.,

$$a_0 + a_1x + a_2x^2 + \dots + a_nx^n = 0 \text{ for all } x$$

When $n = 1$, we have $a_0 + a_1x = 0$ for all x . Setting $x = 0$, we obtain $a_0 = 0$. Then, from $a_1x = 0$, obviously we have $a_1 = 0$.

When $n = 2$, we set $x = 0$ and obtain $a_0 = 0$. Then, we have $x(a_1 + a_2x) = 0$ for all x . Hence, $a_1 + a_2x$ must be 0 and $a_1 = a_2 = 0$ according to the case $n = 1$. In conclusion, the coefficients $(a_0, a_1, \dots, a_n)$ have to be 0 for any integer n .

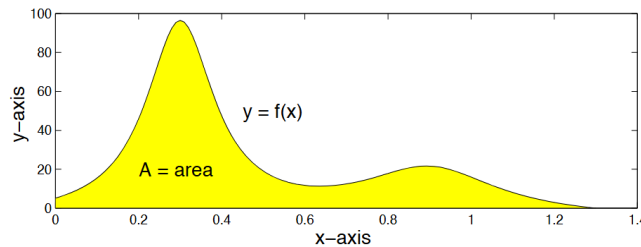
Chapter 6

Numerical integration

Numerical integration (or numerical quadrature) is to calculate integrals numerically,

$$I = \int_a^b f(x)dx$$

= the area under the curve of f in the interval $[a, b]$ (if f is positive)



Mathematical example

Example of an integral, which can be calculated analytically,

$$\int_0^1 e^{-x} dx = -e^{-x} \Big|_0^1 = \left(\frac{1}{2.7183} - 1 \right) = 0.6321$$

Example of an integral, which cannot be expressed in terms of elementary functions,

$$\phi = \int_0^1 \exp(-x^2) dx$$
$$\psi = \int_a^{\pi/2} \sqrt{1 + \cos(x)^2} dx$$

The integral ϕ commonly appears in probability theory, and the integral ψ commonly appears in analytic geometry.

The numerical problem

To determine the numerical value of an integral,

$$I = \int_a^b f(x) dx$$

We can calculate an approximation through,

$$I \approx \int_a^b f_n(x) dx$$

Where f_n is a polynomial that approximates f , we can integrate f_n analytically, since its integral has a "closed" expression. In this way, we obtain an approximation to the integral of f .

The main difference between different numerical integration algorithms is how to choose this "approximate" polynomial.

6.1 Approximation through Interpolating polynomials

x	x_0	x_1	$\cdots$	x_n
y	$f(x_0)$	$f(x_1)$	$\cdots$	$f(x_n)$

For a table of the form of Table ??, the Lagrange form of the interpolating polynomial is $p_n(x) = \sum_{i=0}^n l_i(x)f(x_i)$.

Hence, the approximation of integration f is,

$$\begin{aligned} I &= \int_a^b f(x)dx \approx \int_a^b p_n dx = \int_a^b \sum_{i=0}^n l_i(x)f(x_i)dx \\ &= \sum_{i=0}^n f(x_i) \int_a^b l_i(x)dx = \sum_{i=0}^n f(x_i)A_i \end{aligned}$$

Where the constants, $A_i = \int_a^b l_i(x)dx$ for $i = 0, \dots, n$, can be predetermined.

The approximation of the integral is just a weighted linear combination of the function values $f(x_0), f(x_1), \dots, f(x_n)$.

Not much calculation is needed!

6.2 Newton-Cotes formulas

For $n + 1$ equally spaced nodes $x_i = a + ih$, where $h = (b - a)/n$, with predetermined A_i we obtain the Newton-Cotes quadrature formula. We can estimate the error through the first/second interpolation error Theorem in Section 5.7. Using the Second Interpolation Error Theorem we have,

For all $x \in [a, b]$, we have,

$$\left| f(x) - p_n(x) \right| \leq \frac{1}{4(n+1)} \left| f^{(n+1)}(\xi) \right| h^{n+1} \leq \frac{1}{4(n+1)} M h^{n+1}$$

Thus,

$$\left| \int_a^b f(x)dx - \int_a^b p_n(x)dx \right| \leq \int_a^b \left| f(x) - p_n(x) \right| dx \leq (b-a) \frac{1}{4(n+1)} M h^{n+1}$$

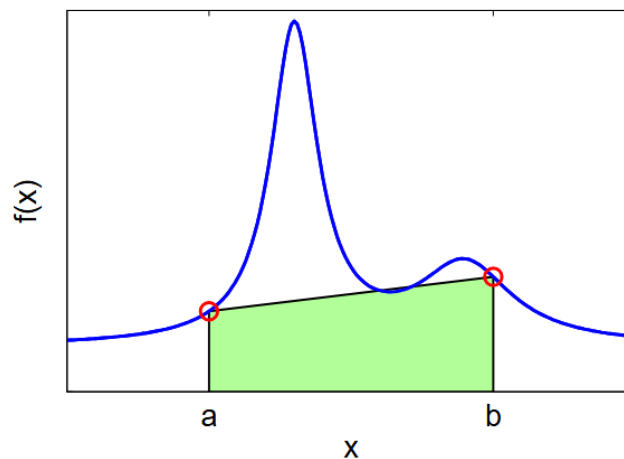
But we can get even better error estimation for a specific n .

Increasing the degree m of the polynomial using Newton-Cote formulas, does not make the error decrease since the error does not decrease monotonically with m

6.3 Trapezoid rule

For $n=1$ we choose $x_0 = a$ and $x_1 = b$.

The approximation by a polynomial of degree 1, p_1 , is in fact the line through the points $(a, f(a))$ and $(b, f(b))$ like the plot below as an example,



$$\begin{aligned}
\int_a^b f(x)dx &\approx \int_a^b p_1(x)dx = \int_a^b \left(f(a)l_0(x) + f(b)l_1(x) \right) dx \\
&= f(a) \int_a^b l_0(x)dx + f(b) \int_a^b l_1(x)dx \\
&= f(a) \frac{b-a}{2} + f(b) \frac{b-a}{2} + E_t \\
&= h \frac{1}{2} \left(f(a) + f(b) \right) + E_t
\end{aligned}$$

Where $h = (b - a)$ and E_t is the truncation error we show further down,

$$E_t = -\frac{1}{12}h^3 f''(\xi)$$

Where ξ is an unknown value between a and b . Thus,

$$|E_t| \leq \frac{1}{12}h^3 \max_{a \leq \xi \leq b} |f''(\xi)|$$

Hence, the error is large if $|f''(\xi)|$ is large, or h is large. We must balance a large $|f''(\xi)|$ with a small spacing h

6.3.1 Error analysis for trapezoid rule

In the book, it used the first interpolation error theorem from Section 5.7,

$$\begin{aligned}
f(x) - p_n(x) &= \frac{1}{(n+1)!} f^{(n+1)}(\xi) \prod_{i=0}^n (x - x_i) \\
f(x) - p_1(x) &= \frac{1}{(1+1)!} f^{(1+1)}(\zeta(x)) (x-a)(x-b)
\end{aligned}$$

To analyze the error,

$$\begin{aligned}
\int_a^b f(x)dx - \int_a^b p_1(x)dx &= \int_a^b \frac{1}{2} f''(\zeta(x)) (x-a)(x-b) dx \\
&= \frac{1}{2} f''(\xi) \int_a^b (x-a)(x-b) dx \\
&= -\frac{1}{12} (b-a)^3 f''(\xi), \text{ where } a < \xi < b
\end{aligned}$$

A difficulty in this proof is that we know nothing about $\zeta(x)$, which can be discontinuous. But it can be shown (not to be learnt how), that the function $x \rightarrow f''(\zeta(x))$ is indeed continuous and thus integrable. Then, based on the mean-value theorem for integrals (which is not necessary to know) we get the second equivalence.

6.3.2 Error analysis for trapezoid rule by Taylor series

Recalling the Taylor series,

$$f(a+h) = f(a) + hf'(a) + \frac{h^2}{2!} f''(a) + \dots$$

Thus the Taylor series of Trapezoid rule is,

$$\begin{aligned}
\frac{h}{2} \left(f(a) + f(a+h) \right) &= \frac{h}{2} \left(2f(a) + hf'(a) + \frac{h^2}{2!} f''(a) + \dots \right) \\
&= hf(a) + \frac{h^2}{2} f'(a) + \frac{h^3}{4} f''(a) + \dots
\end{aligned}$$

Taylor series $F(t) = \int_a^t f(x)dx$ is,

$$F(a+h) = F(a) + hF'(a) + \frac{h^2}{2!}F''(a) + \frac{h^3}{3!}F'''(a) + \dots$$

We have $F(a) = 0$, $F'(a) = f(a)$, $F''(a) = f'(a)$ and $F'''(a) = f''(a)$ so,

$$F(a+h) = hf(a) + \frac{h^2}{2!}f'(a) + \frac{h^3}{3!}f''(a) + \dots$$

Now comparing both Taylor series,

$$F(a+h) = \int_a^b f(x)dx = hf(a) + \frac{h^2}{2!}f'(a) + \frac{h^3}{3!}f''(a) + \dots$$

$$\frac{h}{2} \left(f(a) + f(a+h) \right) = hf(a) + \frac{h^2}{2}f'(a) + \frac{h^3}{4}f''(a) + \dots$$

This shows the error is,

$$\begin{aligned} \int_a^b f(x)dx - \frac{h}{2} \left(f(a) + f(a+h) \right) &= \left(\frac{1}{6} - \frac{1}{4} \right) h^3 f''(a) + \dots = \\ &= -\frac{1}{12}(b-a)^3 f''(a) + \dots \end{aligned}$$

We recognize the constant $\frac{1}{12}(b-a)^3$ from before.

6.4 Composite trapezoid rule

We set $h = (b-a)/n$ and $x_i = a + ih$ and apply trapezoid rule to every subinterval $[x_{i-1}, x_i]$,

$$\int_{x_{i-1}}^{x_i} f(x)dx = \frac{h}{2}(f_{i-1} + f_i) - \frac{h^3}{12}f''(\xi_i), \text{ where } x_{i-1} < \xi < x_i$$

Thus we have,

$$\int_a^b f(x)dx = h \left(\frac{1}{2}f_0 + f_1 + f_2 + \dots + f_{n-1} + \frac{1}{2}f_n \right) + E_t$$

Where the truncated error,^o

$$E_t = -\frac{(b-a)h^2}{12} \left(\frac{1}{n} \sum_{i=1}^n f''(\xi_i) \right)$$

With, $a \leq x_{i-1} < \xi_i < x_i \leq b$.

Since f'' is continuous, we know that the average,

$$\left(\frac{1}{n} \sum_{i=1}^n f''(\xi_i) \right)$$

lies between the smallest and largest values of f'' on the interval $]a, b[$. By the intermediate-value theorem, there is a point $\zeta \in]a, b[$ such that,

$$\left(\frac{1}{n} \sum_{i=1}^n f''(\xi_i) \right) = f''(\zeta)$$

Theorem 6.4.1. Precision of composite trapezoid rule If f'' exists and is continuous on the interval $[a, b]$ and if the interval is divided into n equal sub intervals with points $x_i = a + ih$ and spacing $h = (b-a)/n$, then the composite trapezoid rule is,

$$\int_a^b f(x)dx = h \left(\frac{1}{2}f_0 + f_1 + f_2 + \dots + f_{n-1} + \frac{1}{2}f_n \right) + E_t$$

Where the error term is,

$$E_t = -\frac{(b-a)h^2}{12} \left(\frac{1}{n} \sum_{i=1}^n f''(\xi_i) \right) = -\frac{(b-a)h^2}{12} f''(\zeta)$$

With $a < \zeta < b$.

The error is $\mathcal{O}(h^2)$, i.e., it is proportional to h^2 and $\frac{1}{n^2}$

Doing it in Python as,

```
def CompositeTrapezoidalRule(f, a, b, n):
    h = (b - a) / n
    integral = 0.5 * (f(a) + f(b))

    for i in range(1, n):
        integral += f(a + i * h)

    integral *= h
    return integral
```

And for a given table of nodes and values,

```
def CompositeTrapezoidalRuleForTable(x, values):
    n = len(x) - 1
    h = (x[-1] - x[0]) / n
    integral = 0.5 * (values[0] + values[-1])

    for i in range(1, n):
        integral += values[i]

    integral *= h
    return integral
```

6.5 Simpson's rule

For a table of the form ??, where $x_0 = a$, $x_2 = b$ and x_1 is chosen as the midpoint $\frac{a+b}{2}$, the Lagrange form of the interpolating polynomial is $p_2(x) = \sum_{i=0}^2 l_i(x)f(x_i)$.

Hence, the corresponding approximation of the integral of f is,

$$\begin{aligned} I &= \int_a^b f(x)dx \approx \int_a^b p_2(x)dx = \int_a^b \sum_{i=0}^2 l_i(x)f(x_i)dx \\ &= \sum_{i=0}^2 f(x_i) \int_a^b l_i(x)dx = \frac{h}{3} \left(f(x_0) + 4f(x_1) + f(x_2) \right), \text{ where } h = \frac{b-a}{2} \end{aligned}$$

The weights of the function values $f(x_0), f(x_1), f(x_2)$ can be calculated directly or be derived as follows, Let $a = -1$ and $b = 1$, and consider the formula,

$$\int_{-1}^1 f(x)dx \approx Af(-1) + B(f(0) + Cf(1))$$

This formula must give correct values for the integral whenever f is a polynomial of degree at most 2, i.e., it integrates 1, x and x^2 correctly. We use this to determine the coefficients A , B and C ,

$$\int_{-1}^1 1dx = 2 = A + B + C$$

$$\int_{-1}^1 xdx = 0 = -A + 0B + C$$

$$\int_{-1}^1 x^2 dx = \frac{2}{3} = A + 0B + C$$

The solution is $(A, B, C) = (1/3, 4/3, 1/3)$.

Using the linear mapping $y = \frac{1}{2}(b-a)x + \frac{1}{2}(a+b)$ from $[-1, 1]$ to $[a, b]$, we obtain the general formula previously found,

$$\sum_{i=0}^2 f(x_i) \int_a^b l_i(x) dx = \frac{h}{3} \left(f(x_0) + 4f(x_1) + f(x_2) \right), \text{ where } h = \frac{b-a}{2}$$

6.5.1 Error of Simpson's rule

Simpson's rule is designed to give correct values for the integral of a polynomial of degree at most 2. We can apply Taylor series to obtain the error term of Simpson's rule,

$$E_t = -\frac{h^5}{90} f^{(4)}(\xi) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi)$$

Where $f^{(4)}$ is the fourth order derivative of f . **Simpson's rule also gives correct values for the integral of the polynomial of degree 3!**

6.6 Composite Simpson's rule

Similar to the composite trapezoid rule (defined previously), we can divide $[a, b]$ into N sub intervals, each of width $h = (b-a)/n$ and write,

$$\begin{aligned} \int_a^b f(x) dx &= \int_{x_0}^{x_2} f(x) dx + \int_{x_2}^{x_4} f(x) dx + \cdots + \int_{x_{n-2}}^{x_n} f(x) dx \\ &= \sum_{i=0}^{n/2} \left(\frac{h}{3} (f_{2i-2} + 4f_{2i-1} + f_{2i}) - \frac{h^5}{90} f^{(4)}(\eta_i) \right) \\ &= \frac{h}{3} (f_0 + 4f_1 + 2f_2 + 4f_3 + \cdots + 3f_{n-2} + 4f_{n-1} + f_n) + E_t \end{aligned}$$

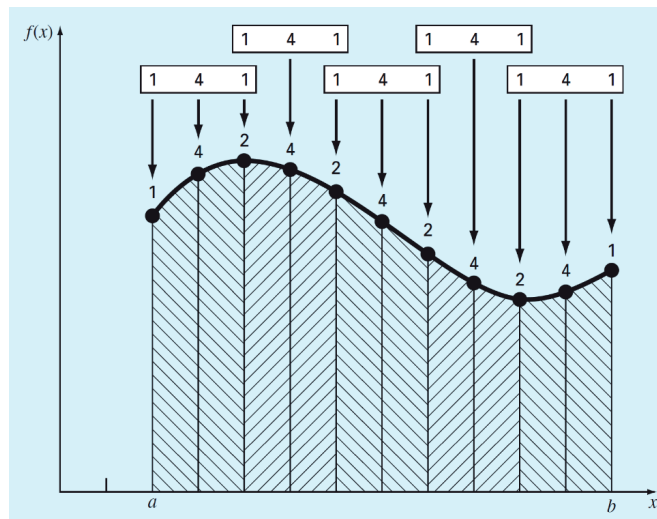
Where we again apply the intermediate-value theorem on the average of $f^{(4)}$ and obtain the error,

$$E_t = -\frac{b-a}{180} h^4 f^{(4)}(\xi) = -\frac{(b-a)^5}{180n^4} f^{(4)}(\xi), \text{ where } a < \xi < b$$

The error is proportional with h^4 and $\frac{1}{n^4}$.

n must be even!

Weights in composite Simpson's rule follow the following graph,



Doing it in Python as,

```

def CompositeSimpsonsRule(f, a, b, n):
    if n % 2 == 1:
        raise ValueError("n must be even for Simpson's rule.")
    h = (b - a) / n
    integral = f(a) + f(b)

    for i in range(1, n, 2):
        integral += 4 * f(a + i * h)
    for i in range(2, n-1, 2):
        integral += 2 * f(a + i * h)

    integral *= h / 3
    return integral

```

And for a given table of nodes and values,

```

def CompositeSimpsonsRuleForTable(x, values):
    n = len(x) - 1
    if n % 2 == 1:
        raise ValueError("Number of intervals must be even for Simpson's rule.")
    h = (x[-1] - x[0]) / n
    integral = values[0] + values[-1]

    for i in range(1, n, 2):
        integral += 4 * values[i]
    for i in range(2, n-1, 2):
        integral += 2 * values[i]

    integral *= h / 3
    return integral

```

6.7 When the Polynomial-based formula is effective and accurate

Until now, all methods based on polynomial interpolation on equally spaced nodes. Therefore we can conclude,

The method is effective when the integrand f resembles a polynomial

For functions that don't resemble polynomials, we use an adaptive algorithm, where we use sub intervals of different lengths that are suitable for the function. Specifically we use fewer and larger sub intervals where f is "simple" (i.e. varying less), more and smaller sub intervals where f varies a lot.

The partitioning of the interval is automatically determined, the partition is generated adaptively based on the function.

6.7.1 Recursive adaptive integration

Assuming that we have an approximation l_1 with an error E ,

$$\int_a^b f(x)dx = I_i + E$$

If the error E is too large, we divide $[a, b]$ into two sub intervals $[a, c]$ and $[c, b]$ with the midpoint $c = (a + b)/2$,

$$\int_a^b f(x)dx = \int_a^c f(x)dx + \int_c^b f(x)dx = I_L + E_L + I_R + E_R$$

Where E_L and E_R are the error estimates from I_L and I_R , respectively.

If the total error $E_L + E_R$ is sufficiently small, then $I_2 = I_L + I_R$ is an acceptable approximation to the integral. Recursive strategy, use the same idea to approximate I_L and I_R .

Computing the error estimate for Recursive adaptive integration

We assume that $F^{(4)}$ is constant throughout the interval $[a, b]$. In Simpson's rule, with the length of the sub interval $h_1 = h$,

$$I = I(h_1) + Ch^5, \text{ where } C = \text{constant}$$

Similarly, with the length of the interval $h_2 = h_1/2 = h/2$,

$$I = I_L(h_2) + C\left(\frac{h}{2}\right)^5 + I_R(h_2) + C\left(\frac{h}{2}\right)^5 = I(h_2) + \frac{1}{16}Ch^5$$

Note it is the same constant C in both expressions.

According to the error terms in both terms, we have,

$$\begin{aligned} I - I(h_2) &= \frac{1}{16}(I - I(h_1)) \\ I &= \frac{16}{15}\left(I(h_2) - \frac{1}{16}I(h_1)\right) \\ &= I(h_2) + \frac{I(h_2) - I(h_1)}{15} \end{aligned}$$

Therefore, the error estimate is,

$$\frac{I(h_2) - I(h_1)}{15}$$

And this is called Richardson-extrapolation.

The way we do it is,

Start by calculating the approximations $I(h_1)$ and $I(h_2)$ with Simpson's rule and the length of the interval $h_1 = (b - a)/2$ and $h_2 = (b - a)/4$.

Then we use Richardson-extrapolation to calculate a better approximation to the integral,

$$I_3 = I(h_2) + \frac{1}{15}(I(h_2) - I(h_1))$$

and $|I_3 - I(h_2)| = \frac{1}{15}|I(h_2) - I(h_1)|$ is the error estimate for $I(h_2)$

If the error estimate is less than τ (given in advance), then stop. Otherwise, repeat the process on each sub intervals, $[a, c]$ and $[c, b]$, with $c = (a + b)/2$. On each sub interval, the tolerance τ is replaced by $\tau/2$.

6.8 Gaussian Quadrature Theorem

By a special placement of the nodes, the accuracy of the numerical integration could be greatly increases

Theorem 6.8.1. (Gaussian Quadrature) Let q be a non trivial polynomial of degree $n + 1$ such that,

$$\int_a^b x^k q(x) dx = 0. \text{ for } k = 0, 1, \dots, n$$

Let $x_0, x_1, \dots, x_n$ be the zeros of q . Then the formula,

$$\int_a^b f(x) dx \approx \sum_{i=0}^n A_i f(x_i), \text{ where } A_i = \int_a^b l_i(x) dx$$

With these x_i 's as nodes will exact for all polynomials of degree at most $2n + 1$.

Note that we need to precompute all the x_i 's.

6.9 Numerical integration with Scipy

Scipy offers several functions for numerical integration,

```
from scipy.integrate import quad
I = quad(lambda x: np.sin(x), 0, np.pi, epsabs = 1e-1)
# Returns an estimate of the integral
# and an estimate of the absolute error
```

The following functions are for multiple integrals,

```
dblquad()
tplquad()
nquad()
```

And finally for the adaptive Gauss quadrature,

```
quadrature()
```

Chapter 7

Numerical Solution of Differential Equations: Initial-Value Problems

This chapter focuses on solving first-order ordinary differential equations (ODEs) numerically. We study ODEs of the form

$$\frac{dx}{dt} = f(t, x),$$

where f is a scalar function of (t, x) . A numerical solution produces an approximation of the analytical solution $x(t)$ only at discrete grid points

$$t_i = a + ih, \quad i = 0, \dots, n,$$

where $h = (b - a)/n$ is the step size.

7.1 From Higher-Order Equations to First-Order Systems

An n th-order scalar ODE

$$\frac{d^n x}{dt^n} = f\left(t, x, \frac{dx}{dt}, \dots, \frac{d^{n-1}x}{dt^{n-1}}\right)$$

can always be expressed as a system of n coupled first-order equations. Define

$$z_1 = x, \quad z_2 = \frac{dx}{dt}, \quad \dots, \quad z_n = \frac{d^{n-1}x}{dt^{n-1}},$$

which gives

$$\begin{aligned} \frac{dz_1}{dt} &= z_2, \\ \frac{dz_2}{dt} &= z_3, \\ &\vdots \\ \frac{dz_n}{dt} &= f(t, z_1, z_2, \dots, z_n). \end{aligned}$$

In vector form,

$$\frac{d\mathbf{z}}{dt} = \mathbf{f}(t, \mathbf{z}), \quad \mathbf{z} = (z_1, z_2, \dots, z_n)^\top.$$

Note: numerical solvers operate only on first-order systems, so higher-order ODEs must first be rewritten in this form. A system of dimension n requires n auxiliary (initial) conditions to specify a unique solution.

7.2 Initial-Value Problems and Uniqueness

An n th-order ODE determines a unique solution only when n initial conditions are specified.

Example. All functions $x(t) = ke^{-3t}$ satisfy $\dot{x} = -3x$, but an extra condition, e.g. $x(0) = 7$, fixes the unique solution

$$x(t) = 7e^{-3t}.$$

Existence and Uniqueness Theorem. If $f(t, x)$ and its partial derivative $\partial f / \partial x$ are continuous in a neighborhood of (t_0, x_0) , then the IVP $x' = f(t, x)$, $x(t_0) = x_0$ has a *unique* solution in some interval around t_0 . If this condition fails, multiple or no solutions may exist.

Example of Non-Uniqueness. For $x' = x^{1/3}$, $x(0) = 0$, both $x(t) \equiv 0$ and $x(t) = (\frac{2}{3}t)^{3/2}$ are valid solutions.

7.3 Euler's Method

Consider the IVP

$$\frac{dx}{dt} = f(t, x), \quad x(a) = x_a, \quad t \in [a, b].$$

From the first-order Taylor expansion:

$$x(t+h) \approx x(t) + h x'(t) = x(t) + h f(t, x(t)).$$

This yields the **Euler update**:

$$x_{i+1} = x_i + h f(t_i, x_i), \quad x_0 = x(a).$$

Geometrically, each new point lies on the tangent to the previous one; Euler's method follows the slope field in small steps.

Properties.

- The next point always lies on the tangent line at the previous point.
- Points cannot turn sharply if x is highly curved; thus, Euler's method can be inaccurate or unstable for stiff problems.
- It remains useful for illustration, since all more advanced methods can be viewed as generalizations of Euler.

```
import numpy as np
```

```
def MyEuler(dxdt, tspan, x0, n):
    """
    Uses Euler's method to integrate an ODE.

    Input:
        dxdt = function handle to the rhs of the ODE
        tspan = [a, b] interval of integration
        x0    = initial value
        n     = number of steps

    Output:
        t = vector of independent variable
        x = vector of dependent variable values
    """
    a, b = tspan
    t = np.linspace(a, b, n+1)
    h = (b - a) / n
    x = np.zeros(n+1)
```

```

x[0] = x0
for i in range(n):
    x[i+1] = x[i] + dxdt(t[i], x[i]) * h
return t, x

```

Error. Euler has local truncation error $O(h^2)$ and global error $O(h)$.

7.4 Taylor Series Methods

Including higher derivatives improves accuracy. For instance, the 2nd-order Taylor expansion gives

$$x_{i+1} = x_i + hf(t_i, x_i) + \frac{h^2}{2} \left(f_t(t_i, x_i) + f_x(t_i, x_i)f(t_i, x_i) \right).$$

The local truncation error is $\frac{1}{(k+1)!}x^{(k+1)}(\xi)h^{k+1}$ for a k th-order Taylor method.

Remarks.

- Easy to understand and implement.
- Requires f_t and f_x (may use symbolic or automatic differentiation).
- Methods that use only $f(t, x)$ (like Runge–Kutta) are often preferred.

```

import numpy as np

def MyTaylorOrder2(f, dfdx, dfdt, tspan, x0, n):
    """
    Solves x' = f(t,x) using the 2nd-order Taylor series method.
    """
    a, b = tspan
    t = np.linspace(a, b, n+1)
    h = (b - a) / n
    x = np.zeros(n+1)
    x[0] = x0

    for i in range(n):
        ti, xi = t[i], x[i]
        f_val = f(ti, xi)
        x[i+1] = xi + h*f_val + 0.5*h**2 * (
            dfdt(ti, xi) + dfdx(ti, xi) * f_val
        )
    return t, x

```

7.5 Heun's and Midpoint Methods (Runge–Kutta 2)

Two-stage Runge–Kutta methods generalize Euler:

$$\begin{aligned}
 K_1 &= f(t_i, x_i), \\
 K_2 &= f(t_i + \alpha h, x_i + \beta h K_1), \\
 x_{i+1} &= x_i + h(\omega_1 K_1 + \omega_2 K_2).
 \end{aligned}$$

Order Conditions (Why Heun is 2nd Order)

Matching Taylor expansions up to $O(h^2)$ yields

$$\omega_1 + \omega_2 = 1, \quad \omega_2 \alpha = \frac{1}{2}, \quad \omega_2 \beta = \frac{1}{2}.$$

Hence local error $O(h^3)$, global $O(h^2)$. Heun's method satisfies these with $(\alpha, \beta, \omega_1, \omega_2) = (1, 1, \frac{1}{2}, \frac{1}{2})$.

```
import numpy as np

def MyHeun(dxdt, tspan, x0, n):
    """
    Uses Heun's method (RK2) to integrate an ODE.
    """
    a, b = tspan
    t = np.linspace(a, b, n+1)
    h = (b - a) / n
    hhalve = h / 2.0
    x = np.zeros(n+1)
    x[0] = x0
    for i in range(n):
        K1 = dxdt(t[i], x[i])
        K2 = dxdt(t[i+1], x[i] + h * K1)
        x[i+1] = x[i] + (K1 + K2) * hhalve
    return t, x
```

7.6 Runge–Kutta 4th Order (RK4)

One of the most widely used methods:

$$x_{i+1} = x_i + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4),$$

where

$$\begin{aligned} K_1 &= f(t_i, x_i), \\ K_2 &= f(t_i + \frac{h}{2}, x_i + \frac{h}{2}K_1), \\ K_3 &= f(t_i + \frac{h}{2}, x_i + \frac{h}{2}K_2), \\ K_4 &= f(t_i + h, x_i + hK_3). \end{aligned}$$

The local error is $O(h^5)$ and the global error $O(h^4)$.

```
import numpy as np

def MyRK4(dxdt, tspan, x0, n):
    """
    Uses Runge-Kutta 4th order method to integrate an ODE.
    """
    a, b = tspan
    t = np.linspace(a, b, n+1)
    h = (b - a) / n
    hhalf = h / 2.0
    hsix = h / 6.0
    x = np.zeros(n+1)
    x[0] = x0

    for i in range(n):
        K1 = dxdt(t[i], x[i])
```

```

    K2 = dxdt(t[i] + hhalf, x[i] + hhalf * K1)
    K3 = dxdt(t[i] + hhalf, x[i] + hhalf * K2)
    K4 = dxdt(t[i] + h, x[i] + h * K3)
    x[i+1] = x[i] + hsix * (K1 + 2*K2 + 2*K3 + K4)
return t, x

```

Special Case. If $x' = f(t)$ depends only on t , the RK4 update is algebraically identical to Simpson's rule for $\int f(t) dt$.

7.7 Error Comparison and Convergence

Method	Local Error	Global Error
Euler	$O(h^2)$	$O(h)$
Heun / Midpoint (RK2)	$O(h^3)$	$O(h^2)$
RK4	$O(h^5)$	$O(h^4)$
Taylor- k	$O(h^{k+1})$	$O(h^k)$

Empirical Order (EOC) Test. Doubling n (halving h) should reduce the error roughly by a factor 2^p for a method of order p .

```

import numpy as np, math
def EOC(solver, f, exact, nlist):
    rows=[]
    for k,n in enumerate(nlist):
        t,x = solver(f, [0,1], 1.0, n)
        err = abs(x[-1]-exact(1.0))
        if k==0:
            rows.append((n,err,None))
        else:
            eoc = math.log(prev_err/err,2.0)
            rows.append((n,err,eoc))
        prev_err=err
    return rows

```

EOC utilities and error helpers

```

import numpy as np, math

def ErrorAtEndpoint(solver, dxdt, tspan, x0, n, exact, t_eval):
    """Absolute error at t_eval."""
    t, x = solver(dxdt, tspan, x0, n)
    return abs(x[-1] - exact(t_eval))

def EOCTable(solver, dxdt, tspan, x0, n_list, exact, t_eval):
    """Tabulates (n, error, EOC)."""
    rows, prev = [], None
    for n in n_list:
        e = ErrorAtEndpoint(solver, dxdt, tspan, x0, n, exact, t_eval)
        eoc = None if prev is None else math.log(prev/e, 2.0)
        rows.append((n, e, eoc))
        prev = e
    return rows

```

Error curves over time (helper).

```
def ErrorCurvesOverTime(solver, dxdt, tspan, x0, n, exact):
    """Absolute and relative error arrays over time."""
    t, x = solver(dxdt, tspan, x0, n)
    x_ex = np.array([exact(tt) for tt in t])
    abs_err = np.abs(x - x_ex)
    rel_err = abs_err / np.maximum(np.abs(x_ex), 1e-300)
    return t, abs_err, rel_err
```

Preview for systems: Euclidean error norm helper

```
import numpy as np

def SystemErrorCurve(t, Xnum, exact_vec):
    """
    Euclidean error ||x_num - x_exact||_2 at each time step.
    exact_vec(t) must return a vector.
    """
    errs = []
    for ti, xi in zip(t, Xnum):
        errs.append(np.linalg.norm(xi - np.asarray(exact_vec(ti)), 2))
    return np.asarray(errs)
```

7.8 Adaptive Methods

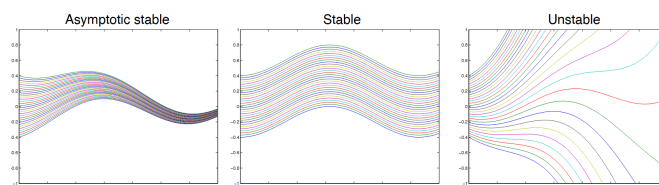
Similar to adaptive integration, one can vary h depending on the estimated local error. The common algorithm in `scipy.integrate.solve_ivp` uses an embedded Runge-Kutta pair (RK45) that computes both 4th and 5th order approximations each step to estimate the error and adjust h automatically.

Advantages.

- Fewer steps for the same accuracy.
- Automatic control of local error within a given tolerance.
- Easily detects regions needing finer resolution.

Limitations. Too large a step size can cause numerical instability even if the analytical solution is stable. Stiff systems require specialized solvers.

7.9 Stability Analysis



Definitions.

- **Stable:** small perturbations in the initial value remain small.
- **Asymptotically stable:** stable and converging to the reference solution.
- **Unstable:** perturbations grow without bound.

Euler Stability Test. For the linear test equation $y' = \lambda y$:

$$y_{i+1} = (1 + h\lambda)y_i.$$

Asymptotic decay requires $|1 + h\lambda| < 1$. If h is too large, numerical divergence occurs even when $\Re(\lambda) < 0$.

Remark. Numerical stability depends not only on the problem but also on the chosen step size h . Stiff problems exhibit strong sensitivity to h and need implicit methods.

Chapter 8

Numerical Solutions of Systems and Boundary-Value Problems

In the previous chapter we studied initial-value problems for a single first-order ODE,

$$\frac{dx}{dt} = f(t, x), \quad x(t_0) = x_0.$$

Now we extend the idea to *systems* of n first-order differential equations and to *boundary-value problems* (BVPs).

8.1 Conceptual Difference Between IVPs and BVPs

An initial-value problem (IVP) specifies all necessary information at a single point, whereas a boundary-value problem (BVP) prescribes conditions at two different points.

As a consequence:

- IVPs can be solved directly by time-stepping methods.
- BVPs generally require iterative methods that adjust unknown initial data to satisfy boundary conditions.

The shooting method converts a BVP into a sequence of IVPs by guessing the missing initial information.

8.2 Systems of First-Order Differential Equations

A system can be written as

$$\frac{d\mathbf{x}(t)}{dt} = \begin{bmatrix} f_1(t, x_1, \dots, x_n) \\ f_2(t, x_1, \dots, x_n) \\ \vdots \\ f_n(t, x_1, \dots, x_n) \end{bmatrix} = \mathbf{f}(t, \mathbf{x}), \quad \mathbf{x} = (x_1, \dots, x_n)^\top.$$

Warning. We will not discuss scaling/normalization of mixed-unit equations (e.g. coulomb vs. m/s). In practice, normalization ensures all variables vary on comparable numerical scales.

8.3 Euler's Method for Systems

For the IVP

$$\mathbf{x}' = \mathbf{f}(t, \mathbf{x}), \quad \mathbf{x}(a) = \mathbf{x}_a, \quad t \in [a, b],$$

the Euler update becomes

$$\mathbf{x}_{i+1} = \mathbf{x}_i + h \mathbf{f}(t_i, \mathbf{x}_i), \quad h = \frac{b-a}{n}.$$

It is the vector analogue of the scalar Euler method and illustrates how all other methods generalize.

8.4 Taylor Series Methods for Systems

Expanding $\mathbf{x}(t + h)$ in a Taylor series gives

$$\mathbf{x}(t + h) = \mathbf{x}(t) + h \mathbf{x}'(t) + \frac{h^2}{2!} \mathbf{x}''(t) + \frac{h^3}{3!} \mathbf{x}^{(3)}(t) + \dots$$

Since h is scalar, the expansion is identical component-wise. Using the chain rule for $\mathbf{x}' = \mathbf{f}(t, \mathbf{x})$:

$$x_i''(t) = \frac{df_i(t, \mathbf{x})}{dt} = \frac{\partial f_i}{\partial t}(t, \mathbf{x}) + \sum_{j=1}^n \frac{\partial f_i}{\partial x_j}(t, \mathbf{x}) x_j'(t),$$

and so on for higher derivatives. A Taylor method of order k includes terms up to h^k and has local error $O(h^{k+1})$, global error $O(h^k)$.

8.5 Fourth-Order Runge–Kutta for Systems

For $\mathbf{x}' = \mathbf{f}(t, \mathbf{x})$,

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \frac{h}{6} (\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4),$$

where

$$\begin{aligned}\mathbf{K}_1 &= \mathbf{f}(t_i, \mathbf{x}_i), \\ \mathbf{K}_2 &= \mathbf{f}(t_i + \frac{h}{2}, \mathbf{x}_i + \frac{h}{2} \mathbf{K}_1), \\ \mathbf{K}_3 &= \mathbf{f}(t_i + \frac{h}{2}, \mathbf{x}_i + \frac{h}{2} \mathbf{K}_2), \\ \mathbf{K}_4 &= \mathbf{f}(t_i + h, \mathbf{x}_i + h \mathbf{K}_3).\end{aligned}$$

The derivation ensures agreement with the Taylor series up to h^4 . Local error $O(h^5)$, global error $O(h^4)$.

```
import numpy as np
```

```
def MyRK4System(odefun, tspan, x0, n):
    """
    Uses Runge-Kutta 4th order method to integrate a system of ODEs.

    Input:
        odefun = function handle returning the rhs vector f(t, x)
        tspan  = [a, b] integration interval
        x0     = initial vector of dependent variables
        n      = number of steps

    Output:
        t = vector of time points
        x = 2D array; each row gives x(t_i)
    """
    a, b = tspan
    t = np.linspace(a, b, n+1)
    h = (b - a) / n
    hhalf, hsix = h/2.0, h/6.0
    x = np.zeros((n+1, len(x0)))
    x[0] = x0
    for i in range(n):
        ti, xi = t[i], x[i]
        K1 = odefun(ti, xi)
        K2 = odefun(ti + hhalf, xi + hhalf*K1)
        K3 = odefun(ti + hhalf, xi + hhalf*K2)
        K4 = odefun(ti + h, xi + h*K3)
        x[i+1] = xi + hsix*(K1 + 2*K2 + 2*K3 + K4)
    return t, x
```

Example (Test System).

$$\mathbf{x}' = \begin{bmatrix} -x_2 \\ x_1 \end{bmatrix}, \quad \mathbf{x}(0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Exact solution $\mathbf{x}(t) = (\cos t, \sin t)^\top$ describes a circle. Comparing Euler and RK4 on this problem shows RK4 attains high accuracy for the same number of function evaluations.

8.6 Converting Higher-Order Equations to First-Order Systems

An n th-order scalar equation

$$\frac{d^n x}{dt^n} = f\left(t, x, \frac{dx}{dt}, \dots, \frac{d^{n-1}x}{dt^{n-1}}\right)$$

can be written as

$$\frac{d\mathbf{z}}{dt} = \begin{bmatrix} z_2 \\ z_3 \\ \vdots \\ z_n \\ f(t, z_1, z_2, \dots, z_n) \end{bmatrix} = \mathbf{f}(t, \mathbf{z}), \quad \mathbf{z} = \begin{bmatrix} x \\ \dot{x} \\ \vdots \\ x^{(n-1)} \end{bmatrix}.$$

This representation allows any ODE of order n to be solved by a first-order system solver.

8.7 Boundary-Value Problems (BVPs)

For a first-order system $\mathbf{x}' = \mathbf{f}(t, \mathbf{x})$:

- **Initial-value problem:** all conditions specified at t_0 .
- **Boundary-value problem:** conditions given at two or more distinct points t_0, t_s .

Unlike IVPs, BVPs may have zero, one, or infinitely many solutions.

8.8 Example BVP

$$x'' = -x, \quad x(0) = 1, \quad x\left(\frac{\pi}{2}\right) = -3.$$

The general solution $x(t) = c_1 \sin t + c_2 \cos t$. Boundary conditions give $c_2 = 1$ and $c_1 = -3$, so

$$x(t) = -3 \sin t + \cos t.$$

8.9 The Shooting Method

To solve a linear 2nd-order BVP

$$y'' = g(x, y, y'), \quad y(a) = \alpha, \quad y(b) = \beta,$$

set

$$z_1 = y, \quad z_2 = y',$$

which yields

$$z_1' = z_2, \quad z_2' = g(x, z_1, z_2).$$

We guess $z_2(a) = z$, integrate the resulting IVP, and define

8.9.1 Interpretation of the Shooting Parameter

The shooting parameter

$$z = y'(a)$$

represents the unknown initial slope required to generate a solution that satisfies the boundary condition at the right endpoint.

Each value of z defines a distinct initial-value problem and therefore a distinct trajectory $y(x; z)$.

The goal of the shooting method is not to solve the differential equation itself, but to determine the specific value of z that produces a trajectory satisfying the boundary condition at $x = b$.

$$\phi(z) = y(b; z).$$

8.9.2 Interpretation of the Shooting Function

The shooting function

$$\phi(z) = y(b; z)$$

maps an initial slope z at $x = a$ to the resulting endpoint value at $x = b$.

Thus:

- Input: a guess for the initial slope z ,
- Output: the value reached at the right boundary.

Solving the BVP is equivalent to solving the scalar nonlinear equation

$$\phi(z) = \beta.$$

This equation is typically solved using root-finding methods such as the secant method or Newton's method.

Adjust z until $\phi(z) = \beta$.

8.9.3 Why Root-Finding Is Required

Although the differential equation is linear or nonlinear in y , the mapping $z \mapsto \phi(z)$ is generally nonlinear.

Therefore, determining the correct shooting parameter requires solving a nonlinear scalar equation.

Root-finding algorithms are applied to $\phi(z) - \beta$, not to the differential equation itself.

If the ODE is linear, $\phi(z)$ is nearly linear in z , so a single secant step suffices.

Example (“Shooting Balls”).

$$\ddot{y} = -g, \quad y(0) = 0, \quad y(10) = 0.$$

General solution $y(t) = -\frac{1}{2}gt^2 + at + b$. Applying the boundaries gives $b = 0$, $a = 5g$, so

$$y(t) = \frac{g}{2}t(10 - t) = \frac{g}{2}x(10 - x).$$

8.9.4 Role of the Step Size in the Shooting Method

The number of steps n used in the numerical integration controls the accuracy of the computed endpoint value $y(b; z)$.

If n is too small:

- the trajectory is inaccurate,
- the shooting function $\phi(z)$ is inaccurate,
- the root-finding method may converge to an incorrect value.

Thus, accuracy of the integration step is a prerequisite for correctness of the shooting method.

8.9.5 Choosing the Step Size

A practical way to choose n is step doubling:

1. Compute $y_n(b)$ using n steps,
2. Compute $y_{2n}(b)$ using $2n$ steps,
3. Increase n until $|y_{2n}(b) - y_n(b)|$ is sufficiently small.

This ensures that the numerical error in the shooting function is negligible compared to the tolerance used in the root-finding method.

Numerical Procedure. If the analytic solution is unknown:

1. Integrate the IVP for two guesses z_1, z_2 .
2. Compute corresponding $\phi(z_1), \phi(z_2)$.
3. Update by the secant method

$$z_3 = z_2 - (\phi(z_2) - \beta) \frac{z_2 - z_1}{\phi(z_2) - \phi(z_1)}.$$

Secant root-finder (utility).

```
def MySecant(phi, z0, z1, tol=1e-10, maxit=50):
    """
    Secant method for scalar roots: find z such that phi(z)=0.
    """
    f0, f1 = phi(z0), phi(z1)
    for k in range(1, maxit+1):
        if abs(f1 - f0) < 1e-300:
            return z1, k
        z2 = z1 - f1*(z1 - z0)/(f1 - f0)
        f2 = phi(z2)
        if abs(f2) < tol:
            return z2, k
        z0, f0, z1, f1 = z1, f1, z2, f2
    return z1, maxit
```

Shooting wrappers (one shot and combined solver).

```
import numpy as np
```

```
def ShootOnce(rhs, tspan, y0, z, n, integrator):
    """
    One 'shot' for BVP: given slope guess z = y'(a),
    integrate and return y(b).
    """
    x0 = np.array([y0, z], dtype=float)
    t, Z = integrator(lambda t, Z: rhs(t, Z), tspan, x0, n)
    return Z[-1, 0], t, Z

def FindShootingSlope(rhs, tspan, y0, yb_target, n, z0, z1, integrator, tol=1e-10):
    """
    Uses secant on phi(z)=y(b;z)-yb_target to find slope z(a).
    """
    def phi(z):
```

```

    yb, _, _ = ShootOnce(rhs, tspan, y0, z, n, integrator)
    return yb - yb_target
z_star, it = MySecant(phi, z0, z1, tol=tol)
yb_star, _, _ = ShootOnce(rhs, tspan, y0, z_star, n, integrator)
return z_star, yb_star, it

```

8.9.6 Sanity Check

After computing the shooting parameter z , the resulting numerical solution must satisfy

$$y(b) \approx \beta.$$

Failure to satisfy the boundary condition indicates insufficient integration accuracy or failure of the root-finding step.

8.10 Interpolation of Discrete Values

Suppose numerical integration yields (t_i, x_i) for $i = 0, \dots, n$. To approximate $x(t)$ at intermediate points:

1. Find k such that $t_{k-1} \leq t < t_k$.
2. Interpolate locally using a few nearest neighbors (linear, quadratic, or cubic).

Using nearby points minimizes interpolation error and numerical instability. In Python one may use `np.interp(t, tgrid, xgrid)` for simple linear interpolation.

8.11 Practical Implementation of a Linear BVP

For the linear equation

$$7y'' + y' - y + x = 0, \quad y(0) = 5, \quad y(20) = 8,$$

we write

$$\begin{aligned} z_1' &= z_2, \\ z_2' &= \frac{1}{7}(-z_2 + z_1 - x). \end{aligned}$$

```

def rhs_bvp(x, z):
    return np.array([
        z[1],
        (-z[1] + z[0] - x)/7.0
    ])

```

Integrate twice with $z_2(0) = -0.8$ and $z_2(0) = -0.9$, record $y(20)$, and use the secant update to hit $y(20) = 8$. Define $\phi(z_0) = y(20; z_0)$ and plot $\phi(z_0)$ versus z_0 . Because the equation is linear, $\phi(z)$ is nearly linear and converges in one step.

8.12 Projectile with Quadratic Drag

We now combine all concepts—system formulation, RK4 integration, interpolation, and error estimation.

8.12.1 Governing Equations

From Newton's second law with quadratic air resistance:

$$\begin{aligned} \ddot{x} &= -\frac{C_d A \rho}{2m} v \dot{x}, \\ \ddot{y} &= -g - \frac{C_d A \rho}{2m} v \dot{y}, \\ v &= \sqrt{\dot{x}^2 + \dot{y}^2}. \end{aligned}$$

Let

$$z_1 = x, \quad z_2 = \dot{x}, \quad z_3 = y, \quad z_4 = \dot{y}.$$

Then

$$\begin{aligned} z_1' &= z_2, \\ z_2' &= -K v z_2, \\ z_3' &= z_4, \\ z_4' &= -g - K v z_4, \end{aligned} \quad K = \frac{C_d A \rho}{2m}.$$

```
import numpy as np

def odesystemkanon(t, z):
    # Constants
    g, m, rho, Cd, d = 9.81, 0.55, 1.2041, 0.5, 0.07
    A = np.pi * d**2 / 4
    K = (Cd * A * rho) / (2*m)
    x, vx, y, vy = z
    v = np.hypot(vx, vy)
    return np.array([
        vx,
        -K*v*vx,
        vy,
        -g - K*v*vy
    ])
```

8.12.2 Simulation Setup

Initial conditions:

$$(x_0, y_0) = (0, 20), \quad v_0 = 500, \quad \theta_0 = 35^\circ,$$

$$(\dot{x}_0, \dot{y}_0) = v_0(\cos \theta_0, \sin \theta_0).$$

Integrate on $t \in [0, 20]$ with $n = 100$ using RK4.

```
x0 = np.array([0.0, 500*np.cos(np.deg2rad(35)),
               20.0, 500*np.sin(np.deg2rad(35))])
t, Z = MyRK4System(odesystemkanon, [0,20], x0, 100)
x, y = Z[:,0], Z[:,2]
```

8.12.3 Finding the Hit Point ($y = 0$)

Locate the first index k with $y_k < 0$ and interpolate nearby points to find the x value where $y = 0$.

```
def hit_point(x, y):
    k = np.argmax(y < 0) # first index where projectile below ground
    idx = slice(max(k-2,0), min(k+3, len(y)))
    return np.interp(0.0, y[idx][::-1], x[idx][::-1])
```

8.12.4 Mesh Refinement Error Estimate

Compute the hit distance for $n = 25, 50, 100, 200, 400$ and estimate

$$E_{\text{est}}(n) = |x_{\text{hit}}(n) - x_{\text{hit}}(2n)|.$$

Choose the smallest n giving acceptable E_{est} .

8.12.5 Hitting a Target at $(1000, 0)$

Define $\Psi(\theta) = x_{\text{hit}}(\theta) - 1000$ and find the root of Ψ by bisection or secant in $\theta \in [20^\circ, 45^\circ]$. Each evaluation integrates the ODE once with angle θ .

8.12.6 Average Height over First 10 s

Approximate

$$\frac{1}{10} \int_0^{10} y(t) dt$$

using the composite Simpson rule (even number of subintervals) or the trapezoidal rule:

$$\int_0^{10} y(t) dt \approx \sum_i \frac{1}{2}(t_{i+1} - t_i)(y_{i+1} + y_i).$$

8.13 Summary

- All ODEs can be expressed as first-order systems.
- Numerical methods for systems mirror the scalar case.
- BVPs are solved via shooting: integrate an IVP and root-find on the missing initial slope.
- Interpolation allows continuous approximations from discrete data.
- Complex physical models (e.g. projectile with drag) combine these tools seamlessly.

Chapter 9

Linear system

Given n linear equations with n variables,

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\&\vdots \\a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n\end{aligned}$$

In matrix form: $\mathbf{Ax} = \mathbf{b}$ with

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}.$$

If $\mathbf{A}$ is invertible, we can solve $\mathbf{Ax} = \mathbf{b}$ in Python with

```
x = np.linalg.solve(A, b)
```

9.1 Efficiency

From Cramer's rule, the i th component in $\mathbf{x}$ can be written as

$$x_i = \frac{\det(\mathbf{A}^{[i]})}{\det(\mathbf{A})}, \quad i = 1, 2, \dots, n,$$

where $\mathbf{A}^{[i]}$ is obtained from $\mathbf{A}$ by replacing the i th column with the vector $\mathbf{b}$:

$$\mathbf{A}^{[i]} = [A(:, 1 : i - 1), \mathbf{b}, A(:, i + 1 : n)].$$

The computational complexity of calculating a determinant by expansion is

$$\approx n! \quad \text{multiplications.}$$

Gaussian elimination, on the other hand, only requires

$$\approx n^3 \quad \text{multiplications.}$$

Example:

$$n = 10 \Rightarrow (n + 1)n! = 3\,9916\,800, \quad n^3 = 1\,000.$$

9.2 Back substitution for upper triangular systems

A matrix $\mathbf{A}$ is *upper triangular* if

$$a_{ij} = 0, \quad \forall i > j.$$

Example:

$$\begin{bmatrix} 5 & -5 & 10 \\ 0 & 2 & 4 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -25 \\ 16 \\ -2 \end{bmatrix}$$

can be solved by

$$\begin{aligned} x_3 &= (-2)/(-1) = 2, \\ x_2 &= \frac{16 - 4x_3}{2} = \frac{16 - 4 \cdot 2}{2} = 4, \\ x_1 &= \frac{-25 - (-5)x_2 - 10x_3}{5} = \frac{-25 + 5 \cdot 4 - 10 \cdot 2}{5} = -5. \end{aligned}$$

In general,

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \cdots & a_{2n} \\ & & \ddots & \vdots \\ & & & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}.$$

9.2.1 Algorithm

```
def back_substitution(A, b):  
    """  
    Solve Ax = b for x when A is an upper triangular matrix.  
    """  
    import numpy as np  
  
    A = np.array(A, dtype=float)  
    b = np.array(b, dtype=float)  
    n = len(b)  
  
    x = np.zeros(n)  
  
    # Step 1: compute x_n  
    x[n-1] = b[n-1] / A[n-1, n-1]  
  
    # Step 2: loop i = n-2 down to 0  
    for i in range(n-2, -1, -1):  
        s = b[i]  
        for j in range(i+1, n):  
            s -= A[i, j] * x[j]  
        x[i] = s / A[i, i]  
  
    return x
```

9.2.2 Computational complexity

A *LOp* (long operation) is a multiplication or a division.

The number of LOps in back substitution is

$$\begin{aligned} \text{LOps} &= 1 + 2 + \cdots + (n-1) + n \\ &= \frac{1}{2}n(n+1) \\ &= \frac{1}{2}n^2 + \frac{1}{2}n \approx \frac{1}{2}n^2. \end{aligned}$$

The computational complexity is dominated by the highest power of n , so the dominant cost is $1/2n^2$ LOps.

n	$1/2n^2$	$1/2n^2 + 1/2n$
10	50	55
100	5 000	5 050
1 000	500 000	500 500

9.3 Gaussian elimination without pivoting

The goal is to transform the system $\mathbf{Ax} = \mathbf{b}$ into an upper triangular one by creating zeros below the main diagonal. At step k , the k th row is the *pivot row*. For each row $i > k$ we compute the multiplier

$$m_{ik} = \frac{a_{ik}}{a_{kk}}, \quad k < i,$$

and perform the row operations

$$a_{ij} \leftarrow a_{ij} - m_{ik}a_{kj}, \quad b_i \leftarrow b_i - m_{ik}b_k,$$

for all columns $j = k + 1, \dots, n$. This eliminates a_{ik} and continues until the matrix becomes upper triangular. Then we apply back substitution.

9.3.1 Example

Consider the system

$$\begin{bmatrix} 5 & -5 & 10 \\ 2 & 0 & 8 \\ 1 & 1 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -25 \\ 6 \\ 9 \end{bmatrix}.$$

Forward elimination. First eliminate the entries below a_{11} :

$$(2') = (2) - \frac{2}{5}(1), \quad (3') = (3) - \frac{1}{5}(1),$$

giving

$$\begin{bmatrix} 5 & -5 & 10 \\ 0 & 2 & 4 \\ 0 & 2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -25 \\ 16 \\ 14 \end{bmatrix}.$$

Next eliminate the entry below a_{22} :

$$(3'') = (3') - 1 \cdot (2'),$$

yielding the upper-triangular system

$$\begin{bmatrix} 5 & -5 & 10 \\ 0 & 2 & 4 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -25 \\ 16 \\ -2 \end{bmatrix}.$$

Back substitution.

$$x_3 = \frac{-2}{-1} = 2, \quad x_2 = \frac{16 - 4x_3}{2} = 4, \quad x_1 = \frac{-25 + 5x_2 - 10x_3}{5} = -5.$$

Thus

$$x = \begin{bmatrix} -5 \\ 4 \\ 2 \end{bmatrix}.$$

9.3.2 Algorithm

```
def gaussian_elimination(A, b):  
    """  
    Perform Gaussian elimination (no pivoting).  
    A is an n×n matrix, b is an n-vector.  
    Returns the modified (upper triangular) A and b.  
    """  
    import numpy as np  
  
    A = np.array(A, float)  
    b = np.array(b, float)  
    n = len(b)  
  
    for k in range(n - 1):  
        for i in range(k + 1, n):  
            xmult = A[i, k] / A[k, k]  
            for j in range(k + 1, n):  
                A[i, j] = A[i, j] - xmult * A[k, j]  
            b[i] = b[i] - xmult * b[k]  
  
    return A, b
```

9.4 LU-factorization

9.4.1 LU-factorization (no pivoting)

Gaussian elimination without pivoting can be written as the matrix factorization

$$A = LU,$$

where U is the upper triangular matrix obtained after elimination and L is a unit lower triangular matrix containing the elimination multipliers. In particular,

$$\ell_{ik} = m_{ik} = \frac{a_{ik}}{a_{kk}}, \quad k < i, \quad \ell_{kk} = 1.$$

Unit lower triangular means that all diagonal entries of L equal 1. These notations are very useful when solving several systems with the same matrix A but different right-hand sides $\mathbf{b}$.

Each elimination step can be represented by an elementary matrix M_k such that $M_k A$ performs the k th elimination. The full elimination process satisfies

$$M_{n-1} \cdots M_2 M_1 A = U.$$

Taking inverses gives

$$A = M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1} U = LU,$$

where $L = M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1}$ is unit lower triangular and contains the multipliers m_{ik} .

9.4.2 LU-factorization with pivoting

When pivoting is used during Gaussian elimination, the row interchanges can be represented by a permutation matrix P . The resulting factorization becomes

$$PA = LU,$$

where

- P is a permutation matrix encoding the row swaps,

- U is the upper triangular matrix obtained after elimination,
- L is a unit lower triangular matrix containing the elimination multipliers, with

$$\ell_{ik} = m_{ik}, \quad k < i, \quad \ell_{kk} = 1.$$

A permutation matrix P is obtained by permuting the rows of the identity matrix. Each row and each column contains exactly one entry equal to 1 and all other entries equal to 0. Multiplying PA simply reorders the rows of A according to the same permutation.

The only difference from the no-pivoting case is the presence of the permutation matrix P , which guarantees that Gaussian elimination avoids zero pivots and improves numerical stability.

9.4.3 Solving a linear system using the LU-factorization

Given the factorization

$$PA = LU,$$

we can solve $Ax = b$ efficiently in three steps:

1. Permute the right-hand side:

$$\hat{b} = Pb.$$

2. Solve the lower triangular system

$$Ly = \hat{b} \quad \Rightarrow \quad y = L^{-1}\hat{b}$$

by forward substitution.

3. Solve the upper triangular system

$$Ux = y \quad \Rightarrow \quad x = U^{-1}y$$

by back substitution.

Thus the solution is

$$x = U^{-1}L^{-1}Pb.$$

Once L and U are available, it only costs n^2 long operations to solve a new system $Ax = b$ with a different right-hand side.

LU factorization in Python

In practice, LU factorization is obtained directly from `scipy.linalg.lu`:

```
import numpy as np
from scipy.linalg import lu
```

```
A = np.array([
    [0.1,  7.0, -0.3],
    [0.3, -0.2, 10.0],
    [3.0, -0.1, -0.2]
])
```

```
P, L, U = lu(A)
```

Here P is the permutation matrix, L is unit lower triangular, and U is upper triangular such that

$$PA = LU.$$

9.5 Pivoting

9.5.1 Mathematical reason

Example:

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

The multiplier

$$\text{xmult} = a_{21}/a_{11} = 1/0 = \infty$$

and Gaussian elimination breaks down.

Since $\mathbf{A}$ is nonsingular the rows are interchangeable, so

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$x_2 = 1, \quad x_1 = (2 - 1 \cdot 1) = 1.$$

9.5.2 Numerical reason

Solving by Gaussian elimination, but with different numbers of significant digits,

$$\begin{bmatrix} 0.0003 & 3.0000 \\ 1.0000 & 1.0000 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2.0001 \\ 1.0000 \end{bmatrix}$$

digits	x_1	x_2	rel. error in x_1
3	-3.00	0.667	-11
4	0.0000	0.6667	1
5	0.30000	0.66667	0.1
6	0.330000	0.666667	0.01
7	0.3330000	0.6666667	0.001
∞	0.333333...	0.666666...	

Quite a large error, even with many significant digits.

Now with pivoting,

digits	x_1	x_2	rel. error in x_1
3	0.333	0.667	0.001
4	0.3333	0.6667	0.0001
5	0.33333	0.66667	0.00001
6	0.333333	0.666667	0.000001
7	0.3333333	0.6666667	0.0000001
∞	0.333333...	0.666666...	

Much smaller error, even with few significant digits.

9.5.3 Risk of loss of significance without pivoting

Let $\epsilon \ll 1$ be a small number (e.g. $\epsilon = 0.0003$):

$$\begin{bmatrix} \epsilon & 3 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2.0001 \\ 1 \end{bmatrix} \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 2/3 \end{bmatrix}.$$

Gaussian elimination without pivoting:

$$\begin{aligned} \text{xmult} &= \frac{a_{21}}{a_{11}} = \frac{1}{\epsilon} \gg 1, \\ a'_{22} &= a_{22} - \text{xmult} \cdot a_{12} = 1 - \frac{1}{\epsilon} \cdot 3 \approx -\frac{3}{\epsilon}, \\ b'_2 &= b_2 - \text{xmult} \cdot b_1 = 1 - \frac{1}{\epsilon} \cdot 2.0001 \approx -\frac{2}{\epsilon}. \end{aligned}$$

Applying Gaussian elimination without pivoting, we get

$$\mathbf{A}' = \begin{bmatrix} a_{11} & a_{12} \\ 0 & a'_{22} \end{bmatrix} = \begin{bmatrix} \epsilon & 3 \\ 0 & 1 - 3/\epsilon \end{bmatrix}, \quad \mathbf{b}' = \begin{bmatrix} 2.0001 \\ 1 - 2.0001/\epsilon \end{bmatrix}.$$

With very small ϵ , the computer actually calculates

$$\hat{\mathbf{A}}' = \begin{bmatrix} \epsilon & 3 \\ 0 & -3/\epsilon \end{bmatrix}, \quad \hat{\mathbf{b}}' = \begin{bmatrix} 2.0001 \\ -2.0001/\epsilon \end{bmatrix}.$$

Then the absolute error is

$$\mathbf{A}' - \hat{\mathbf{A}}' = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{b}' - \hat{\mathbf{b}}' = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Note: Errors in $\mathbf{A}$ and $\mathbf{b}$ are of the same order.

Calculating the solution,

$$\begin{aligned} \hat{x}_2 &= \hat{b}'_2 / \hat{a}'_{22} \\ &= (-2.0001/\epsilon) / (-3/\epsilon) = 2.0001/3 \approx 2/3, \\ \hat{x}_1 &= (b_1 - a_{12}\hat{x}_2) / a_{11} \\ &= (2.0001 - 3 \cdot 2.0001/3) / \epsilon \approx 0. \end{aligned}$$

The result for $\hat{x}_2$ is OK, but $\hat{x}_1$ is completely wrong. The problem is due to loss of significance in subtraction.

Note: Avoid elements in the matrix increasing in magnitude during Gaussian elimination.

Gaussian elimination with pivoting:

$$\begin{bmatrix} 1 & 1 \\ \epsilon & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2.0001 \end{bmatrix} \quad \text{xmult} = \frac{a_{21}}{a_{11}} = \epsilon.$$

Applying Gaussian elimination,

$$a'_{22} = 3 - \epsilon \cdot 1 \rightarrow \hat{a}'_{22} \approx 3, \quad b'_2 = 2.0001 - \epsilon \cdot 1 \rightarrow \hat{b}'_2 \approx 2.0001.$$

The errors

$$\mathbf{A}' - \hat{\mathbf{A}}' = \begin{bmatrix} 0 & 0 \\ 0 & \epsilon \end{bmatrix}, \quad \mathbf{b}' - \hat{\mathbf{b}}' = \begin{bmatrix} 0 \\ \epsilon \end{bmatrix}$$

are now small.

$$\begin{aligned} \hat{x}_2 &= \hat{b}'_2 / \hat{a}'_{22} = 2.0001/3 \approx 0.6667 \approx 2/3, \\ \hat{x}_1 &= (b_1 - a_{12}\hat{x}_2) / a_{11} = (1 - 1 \cdot 2.0001/3) / 1 \approx 0.3333 \approx 1/3. \end{aligned}$$

It is much better now,

$$\mathbf{x} - \hat{\mathbf{x}} \approx \begin{bmatrix} 3.33 \cdot 10^{-5} \\ -3.33 \cdot 10^{-5} \end{bmatrix}.$$

9.6 Gaussian elimination with pivoting

9.6.1 Gaussian elimination with partial pivoting

At each step k , we choose as pivot the entry in column k with the largest absolute value among the rows $k, \dots, n$. Let ℓ be the index such that

$$|a_{\ell k}| = \max_{i=k, \dots, n} |a_{ik}|.$$

We then interchange rows k and ℓ in A and also interchange b_k and b_ℓ . This guarantees a nonzero (and well-scaled) pivot before continuing with the usual elimination.

Algorithm

```
def gaussian_elimination_partial_pivot(A, b):
    """
    Gaussian elimination with partial pivoting.
    A : (n,n) array_like
    b : (n,) array_like
    Returns: (U, y) where U is upper triangular, y is modified RHS.
    """
    import numpy as np

    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(b)

    for k in range(n - 1):
        # choose pivot row with largest |a_{ik}|, i = k, ..., n-1
        pivot = k + np.argmax(np.abs(A[k:, k]))
        if A[pivot, k] == 0:
            raise ValueError("Matrix is singular (zero pivot).")

        # interchange row k and pivot row in A and in b
        if pivot != k:
            A[[k, pivot]] = A[[pivot, k]]
            b[k], b[pivot] = b[pivot], b[k]

        # elimination below pivot
        for i in range(k + 1, n):
            xmult = A[i, k] / A[k, k]
            for j in range(k + 1, n):
                A[i, j] = A[i, j] - xmult * A[k, j]
            b[i] = b[i] - xmult * b[k]
            A[i, k] = 0.0

    return A, b
```

9.6.2 Gaussian elimination with scaled partial pivoting

There are many strategies for pivoting. Here we introduce another one.

At the beginning of Gaussian elimination, we compute a scale factor s for each equation in the system as

$$s_i = \max_{1 \leq j \leq n} |a_{ij}| = \text{np.max}(\text{np.abs}(A[i, :])), \quad i = 0, 1, \dots, n-1.$$

In step k , we use the equation whose ratio $|a_{\ell k}/s_\ell|$ is the largest as the pivot equation, i.e.

$$\frac{|a_{\ell k}|}{s_\ell} \geq \frac{|a_{ik}|}{s_i}, \quad i = k, \dots, n-1.$$

Algorithm

```
def gaussian_elimination_scaled_pivot(A, b):  
    """  
    Gaussian elimination with scaled partial pivoting.  
    A : (n, n) array_like  
    b : (n,) array_like  
  
    Returns:  
    U, y where U is upper triangular and y is the modified RHS.  
    """  
    import numpy as np  
  
    A = np.array(A, dtype=float)  
    b = np.array(b, dtype=float)  
    n = len(b)  
  
    # Compute scale factors  $s_i = \max_j |a_{ij}|$   
    s = np.max(np.abs(A), axis=1)  
  
    for k in range(n - 1):  
        # choose pivot row using scaled ratios  $|a_k| / s_$   
        ratios = np.abs(A[k:, k]) / s[k:]  
        pivot_offset = np.argmax(ratios)  
        ell = k + pivot_offset  
  
        if A[ell, k] == 0:  
            raise ValueError("Matrix is singular (zero pivot).")  
  
        # swap rows k and ell in A, b, and s  
        if ell != k:  
            A[[k, ell]] = A[[ell, k]]  
            b[k], b[ell] = b[ell], b[k]  
            s[k], s[ell] = s[ell], s[k]  
  
        # elimination below pivot  
        for i in range(k + 1, n):  
            xmult = A[i, k] / A[k, k]  
            A[i, k+1:] = A[i, k+1:] - xmult * A[k, k+1:]  
            b[i] = b[i] - xmult * b[k]  
            A[i, k] = 0.0  
  
    return A, b
```

9.6.3 Computational complexity of "Naive" Gaussian elimination vs Efficient implementation

An example to show the advantage of using LU-factorization for multiple right-hand sides:

$$Ax^{(1)} = b^{(1)}, \quad Ax^{(2)} = b^{(2)}, \quad \dots, Ax^{(p)} = b^{(p)}$$

"Naive" implementation: p Gaussian elimination and p back substitutions. It costs totally $p(n^3/3 + n^2)$ LOps.

Efficient implementation: One LU-factorization with p forward and back substitutions, costs $n^3 + pn^2$ LOps.

Chapter 10

Nonlinear equations

Newton's method can be used to find a root of an equation with a single variable

$$f(x) = 0,$$

and can also be extended to solve systems of nonlinear equations.

10.1 Newton's method for a single equation

For a scalar equation $f(x) = 0$, Newton's method uses the iteration

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - (f'(x_k))^{-1} f(x_k).$$

Derivation

We use the first two terms in the Taylor series to approximate f at $x_{k+1} = x_k + h$:

$$f(x_{k+1}) = f(x_k + h) \approx f(x_k) + hf'(x_k).$$

Setting this approximation equal to zero gives

$$f(x_k) + hf'(x_k) \approx 0 \quad \Rightarrow \quad h = -\frac{f(x_k)}{f'(x_k)}.$$

Thus

$$x_{k+1} = x_k + h = x_k - \frac{f(x_k)}{f'(x_k)}.$$

10.2 Newton's method for a system of nonlinear equations

We now consider a system of three nonlinear equations in three variables:

$$\mathbf{F}(\mathbf{X}) = 0,$$

where

$$\mathbf{X} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad \mathbf{F}(\mathbf{X}) = \begin{bmatrix} f_1(x_1, x_2, x_3) \\ f_2(x_1, x_2, x_3) \\ f_3(x_1, x_2, x_3) \end{bmatrix}.$$

Written explicitly,

$$\begin{aligned} f_1(x_1, x_2, x_3) &= 0, \\ f_2(x_1, x_2, x_3) &= 0, \\ f_3(x_1, x_2, x_3) &= 0. \end{aligned}$$

We start from an initial guess

$$\mathbf{X}^0 = (x_1^0, x_2^0, x_3^0),$$

which is assumed to be close to the solution of the nonlinear system. Let the step be

$$\mathbf{H} = (h_1, h_2, h_3),$$

so that the new point is $\mathbf{X}^0 + \mathbf{H}$.

10.2.1 Linearization via Taylor expansion

Recall the first-order Taylor expansion of f_i at $\mathbf{X}^0$ with step $\mathbf{H}$:

$$\begin{aligned} 0 &= f_1(x_1^0 + h_1, x_2^0 + h_2, x_3^0 + h_3) \\ &\approx f_1(x_1^0, x_2^0, x_3^0) + \frac{\partial f_1}{\partial x_1} h_1 + \frac{\partial f_1}{\partial x_2} h_2 + \frac{\partial f_1}{\partial x_3} h_3, \\ 0 &= f_2(x_1^0 + h_1, x_2^0 + h_2, x_3^0 + h_3) \\ &\approx f_2(x_1^0, x_2^0, x_3^0) + \frac{\partial f_2}{\partial x_1} h_1 + \frac{\partial f_2}{\partial x_2} h_2 + \frac{\partial f_2}{\partial x_3} h_3, \\ 0 &= f_3(x_1^0 + h_1, x_2^0 + h_2, x_3^0 + h_3) \\ &\approx f_3(x_1^0, x_2^0, x_3^0) + \frac{\partial f_3}{\partial x_1} h_1 + \frac{\partial f_3}{\partial x_2} h_2 + \frac{\partial f_3}{\partial x_3} h_3. \end{aligned}$$

We use the first-order Taylor approximation of $\mathbf{F}$ at $\mathbf{X}^0 + \mathbf{H}$ and set it equal to zero.

This gives a *linear* system in the unknowns (h_1, h_2, h_3) :

$$\begin{aligned} \frac{\partial f_1}{\partial x_1} h_1 + \frac{\partial f_1}{\partial x_2} h_2 + \frac{\partial f_1}{\partial x_3} h_3 &= -f_1(x_1^0, x_2^0, x_3^0), \\ \frac{\partial f_2}{\partial x_1} h_1 + \frac{\partial f_2}{\partial x_2} h_2 + \frac{\partial f_2}{\partial x_3} h_3 &= -f_2(x_1^0, x_2^0, x_3^0), \\ \frac{\partial f_3}{\partial x_1} h_1 + \frac{\partial f_3}{\partial x_2} h_2 + \frac{\partial f_3}{\partial x_3} h_3 &= -f_3(x_1^0, x_2^0, x_3^0). \end{aligned}$$

10.2.2 Matrix form and Jacobian

In matrix form this can be written as

$$\begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = - \begin{bmatrix} f_1(x_1^0, x_2^0, x_3^0) \\ f_2(x_1^0, x_2^0, x_3^0) \\ f_3(x_1^0, x_2^0, x_3^0) \end{bmatrix}.$$

More compactly,

$$\underbrace{\mathbf{F}'(\mathbf{X}^0)}_{\text{matrix}} \underbrace{\mathbf{H}}_{\text{vector}} = - \underbrace{\mathbf{F}(\mathbf{X}^0)}_{\text{vector}}.$$

The matrix $\mathbf{F}'(\mathbf{X}^0)$ is called the *Jacobian matrix* of the vector function $\mathbf{F}$ at $\mathbf{X}^0$.

10.3 Newton's iteration for nonlinear systems

For a general nonlinear system $\mathbf{F}(\mathbf{X}) = 0$ in n variables, a Newton iteration from $\mathbf{X}^{(k)}$ proceeds by solving the linear system

$$\mathbf{F}'(\mathbf{X}^{(k)}) \mathbf{H}^{(k)} = -\mathbf{F}(\mathbf{X}^{(k)}),$$

where $\mathbf{F}'(\mathbf{X}^{(k)})$ is the Jacobian matrix at $\mathbf{X}^{(k)}$ and $\mathbf{H}^{(k)}$ is the correction step. If the Jacobian is nonsingular (a singular matrix is a matrix that is not invertible), the new iterate is

$$\mathbf{X}^{(k+1)} = \mathbf{X}^{(k)} + \mathbf{H}^{(k)}.$$

This generalizes the scalar Newton method

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)},$$

since for a system the update can be written as

$$\mathbf{H}^{(k)} = -(\mathbf{F}'(\mathbf{X}^{(k)}))^{-1} \mathbf{F}(\mathbf{X}^{(k)}).$$

Remark: For a system of equations, the scalar quantity $(f'(x_k))^{-1} f(x_k)$ is replaced by the vector expression

$$(\mathbf{F}'(\mathbf{X}^{(k)}))^{-1} \mathbf{F}(\mathbf{X}^{(k)}).$$

10.4 Example: Newton's method for a 2×2 nonlinear system

Consider the system

$$\begin{cases} f_1(x, y) = x^2 + y^2 - 4 = 0, \\ f_2(x, y) = e^x + y - 1 = 0. \end{cases}$$

The Jacobian matrix is

$$F'(x, y) = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{bmatrix} = \begin{bmatrix} 2x & 2y \\ e^x & 1 \end{bmatrix}.$$

Starting with an initial guess $(x^{(0)}, y^{(0)})$, each Newton step solves

$$F'(x^{(k)}, y^{(k)}) \begin{bmatrix} h_1^{(k)} \\ h_2^{(k)} \end{bmatrix} = - \begin{bmatrix} f_1(x^{(k)}, y^{(k)}) \\ f_2(x^{(k)}, y^{(k)}) \end{bmatrix},$$

and updates

$$x^{(k+1)} = x^{(k)} + h_1^{(k)}, \quad y^{(k+1)} = y^{(k)} + h_2^{(k)}.$$

This is the full Newton iteration for a two-variable nonlinear system.

Newton's method in Python

A simple implementation of Newton's method for a system $F(X) = 0$ is

```
import numpy as np
```

```
def Newtonsys(FdF, X0, kmax):
    """Solve system f(x) = 0 by Newton's method.
```

```
    F(x) and F'(x) are computed by the user-supplied function FdF with the
    call [F,dF] = FdF(x), where F is a np.array holding the computed vector F(x) and
    dF is a np.array holding the computed matrix F'(x).
```

```
    The starting point is x0.
```

The function carries out $k_{\max}$ iterations, and stores all $k_{\max}$ iterations as columns on the output matrix $X_{\text{iterations}}$.

Per Christian Hansen and Hans Bruun Nielsen, DTU Compute, April 11, 2016.

Translated to Python spring 2023.

Args:

FdF (function): Function that computes $F(x)$ and $F'(x)$. Must return two `np.array`s.
 X_0 (`np.array`): The starting point
 $k_{\max}$ (`int`): Maximal number of iterations

Returns:

`np.array`: Iterations, stored columnwise
"""

Initialization

$X = X_0$

$Fx, dFx = FdF(X)$

$H = \text{np.linalg.solve}(dFx, Fx)$ *# This is the first step.*

Create the array to store the iterations.

$X_{\text{iterations}} = []$

Now iterate.

for k in `range(1,  $k_{\max}+1$ )`:

$X = X - H$

$X_{\text{iterations}}.\text{append}(X)$

$Fx, dFx = FdF(X)$

$H = \text{np.linalg.solve}(dFx, Fx)$

return `np.array( $X_{\text{iterations}}$ )`

Chapter 11

Factorizations and Sensitivity

This chapter introduces factorizations specialized for symmetric matrices, norm-based tools for measuring errors, and the conditioning of linear systems. These concepts extend the earlier material on solving $Ax = b$ and explain how perturbations propagate through the system.

11.1 Sensitivity of a Simple Root

Before studying sensitivity of linear systems, we recall how sensitivity appears even in the scalar nonlinear case. Consider a function $f(x)$ with a simple root r , meaning

$$f(r) = 0, \quad f'(r) \neq 0.$$

Suppose the function is perturbed to $\tilde{f}(x)$, with

$$|\tilde{f}(x) - f(x)| \leq \delta$$

near the root. Let $\tilde{r}$ be the perturbed root, i.e. $\tilde{f}(\tilde{r}) = 0$.

11.1.1 Error Bound for a Simple Root

If $f'(x)$ does not vanish near r and

$$M = \min_{x \text{ near } r} |f'(x)|,$$

then the perturbation in the root satisfies the bound

$$|r - \tilde{r}| \leq \frac{\delta}{M}.$$

This inequality shows that even a small vertical perturbation of the function can cause a larger horizontal displacement in the root when $f'(r)$ is small.

11.1.2 Interpretation

- If $|f'(r)|$ is large, the root is *stable*: small changes in f produce small changes in r .
- If $|f'(r)|$ is small, the root is *highly sensitive*: even tiny perturbations in the function can shift the root substantially.

This idea is the nonlinear analogue of the bounds we later derive for $Ax = b$, where the sensitivity of the solution depends on $\|A^{-1}\|$ and the condition number $\kappa(A)$.

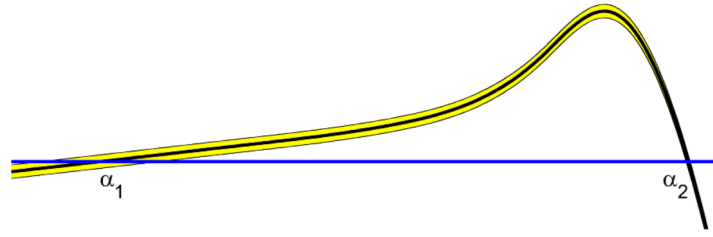


Figure 11.1: Error assessment of a perturbed simple root

11.1.3 Numerical Interpretation (Exam-Oriented)

The bound

$$|r - \tilde{r}| \leq \frac{\delta}{M}$$

shows that root-finding algorithms are sensitive when the derivative at the root is small. In numerical methods such as Newton's method and the secant method, this implies:

- Roots with small $|f'(r)|$ amplify perturbations and roundoff errors.
- Convergence may be slow or unstable even if the algorithm is implemented correctly.
- Error estimates based on $|x_{k+1} - x_k|$ may underestimate the true error.

This explains why numerical root-finding must combine convergence analysis with sensitivity considerations.

11.2 $LDL^\top$ Factorization

For a symmetric matrix $A = A^\top$, the LU factorization satisfies

$$A = LU, \quad A^\top = U^\top L^\top.$$

Because $A = A^\top$, we obtain

$$U(L^\top)^{-1} = L^{-1}U^\top,$$

which is simultaneously upper and lower triangular, hence diagonal. Let this diagonal matrix be D . Then

$$A = LDL^\top.$$

Properties:

- L is unit lower triangular.
- D is diagonal (entries may be positive or negative).
- Computation is cheaper than general LU because symmetry halves the work.

11.2.1 Remarks on Applicability

The $LDL^\top$ factorization exists for any symmetric matrix that admits Gaussian elimination without pivoting. Unlike the Cholesky factorization, the matrix need not be positive definite.

In practice:

- $LDL^\top$ is preferred when symmetry is present but definiteness is unknown.
- Negative entries in D indicate indefiniteness of A .
- For symmetric positive definite matrices, $LDL^\top$ reduces naturally to Cholesky.

```

import numpy as np
from scipy.linalg import lu

A = np.array([
    [0.1, 7.0, -0.3],
    [0.3, -0.2, 10.0],
    [3.0, -0.1, -0.2]
])

# lu(A) willll always give Unit lower triangular matrix
# To make a Unit Upper triangular matrix L_tilde=U.T and U_tilde=L.T
P, L, U = lu(A)
print("Matrix A:")
print(A)
print("\nUnit Lower Triangular Matrix L:")
print(L)
print("\nUpper Triangular Matrix U:")
print(U)

D = np.linalg.inv(L) @ A @ np.linalg.inv(L.T)
print("\nDiagonalized Matrix D:")
print(D)

```

11.3 Cholesky Factorization

A matrix A is *symmetric positive definite* (SPD) if

$$A^T = A \quad \text{and} \quad x^T A x > 0 \quad \forall x \neq 0.$$

For SPD matrices, the diagonal entries of D in the LDL^T factorization are positive, so we can write

$$A = LL^T,$$

with L lower triangular and strictly positive diagonal entries. This is the *Cholesky factorization*. The operations count is $\approx 1/6n^3$, i.e. only half as in LU Factorization

11.3.1 Existence and Failure Cases

The Cholesky factorization exists *if and only if* the matrix A is symmetric positive definite. If A is not positive definite, the algorithm breaks down when attempting to take the square root of a non-positive pivot.

Numerically, failure of Cholesky is therefore a reliable indicator that a matrix is not SPD.

```

A = np.array([[2,1,1], [1,2,-1], [1,-1,3]])

L = cholesky(A, lower=True) # lower triangular L such that A = L L^T

print("Cholesky L:")
print(L)

print("Check A = L L^T:")
print(L @ L.T)
print(A)

print(np.allclose(L @ L.T, A))

```


11.3.2 Example: 2×2 SPD Case

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{12} & a_{22} \end{pmatrix}, \quad L = \begin{pmatrix} \ell_{11} & 0 \\ \ell_{21} & \ell_{22} \end{pmatrix}.$$

From $A = LL^\top$:

$$\ell_{11} = \sqrt{a_{11}}, \quad \ell_{21} = \frac{a_{12}}{\ell_{11}}, \quad \ell_{22} = \sqrt{a_{22} - \ell_{21}^2}.$$

11.3.3 Solving a System via Cholesky

To solve $Ax = b$ using $A = LL^\top$:

1. Solve $Ly = b$ (forward substitution),
2. Solve $L^\top x = y$ (back substitution).

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

c, low = cho_factor(A)    # A = L L^T
x = cho_solve((c, low), b)
```

11.4 Vector and Matrix Norms

11.4.1 Vector 2-Norm

For $x \in \mathbb{R}^n$:

$$\|x\|_2 = \sqrt{x_1^2 + \cdots + x_n^2}.$$

Bounds:

$$\max_i |x_i| \leq \|x\|_2 \leq \sqrt{n} \max_i |x_i|.$$

Absolute and relative errors:

$$\|\tilde{x} - x\|_2, \quad \frac{\|\tilde{x} - x\|_2}{\|x\|_2}.$$

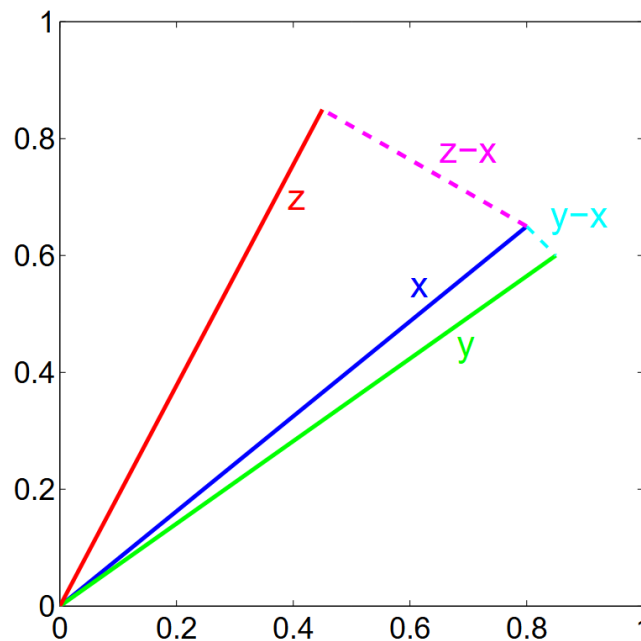


Figure 11.2: Illustration of vector norms and relative error

11.4.2 Matrix 2-Norm

$$\|A\|_2 = \sup_{\|x\|_2=1} \|Ax\|_2,$$

and

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^\top A)}.$$

Interpretation of $\lambda_{\max}(A^\top A)$. In the formula

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^\top A)},$$

the quantity $\lambda_{\max}(A^\top A)$ denotes the *largest eigenvalue of the matrix $A^\top A$* , not the largest eigenvalue of A itself.

This distinction is important:

- For a general matrix A , the eigenvalues of A may be negative or even complex, so they cannot define a norm.
- The matrix $A^\top A$ is always symmetric and positive semidefinite. Therefore all its eigenvalues are real and non-negative.

Since the eigenvalues of $A^\top A$ are the squares of the singular values of A , we have

$$\|A\|_2 = \sigma_{\max}(A),$$

where $\sigma_{\max}(A)$ is the largest singular value.

As a special case:

- If A is symmetric, then $\|A\|_2 = \max_i |\lambda_i(A)|$.
- If A is symmetric positive definite, then $\|A\|_2$ equals the largest (positive) eigenvalue of A .

Useful inequalities:

$$\|Ax\|_2 \leq \|A\|_2 \|x\|_2, \quad \|AB\|_2 \leq \|A\|_2 \|B\|_2.$$

11.4.3 Geometric Interpretation

The matrix 2-norm measures the maximum stretching effect of A on any unit vector:

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2.$$

Thus, $\|A\|_2$ describes how errors or perturbations in the input vector can be magnified by the matrix. This interpretation is central to understanding conditioning and sensitivity.

11.5 Sensitivity of Linear Systems

Consider $Ax = b$. A perturbation in b gives $\tilde{b} = b + \delta b$ and new solution $\tilde{x}$:

$$A\tilde{x} = \tilde{b}.$$

Subtracting,

$$\delta x = \tilde{x} - x = A^{-1} \delta b,$$

thus

$$\|\delta x\|_2 \leq \|A^{-1}\|_2 \|\delta b\|_2.$$

11.5.1 Interpretation

The inequality

$$\|\delta x\|_2 \leq \|A^{-1}\|_2 \|\delta b\|_2$$

shows that even small perturbations in the data vector b can lead to large changes in the solution if $\|A^{-1}\|_2$ is large.

This explains why:

- Ill-conditioned systems are inherently sensitive, regardless of the algorithm used.
- Numerical errors cannot be avoided by improving floating-point accuracy alone.

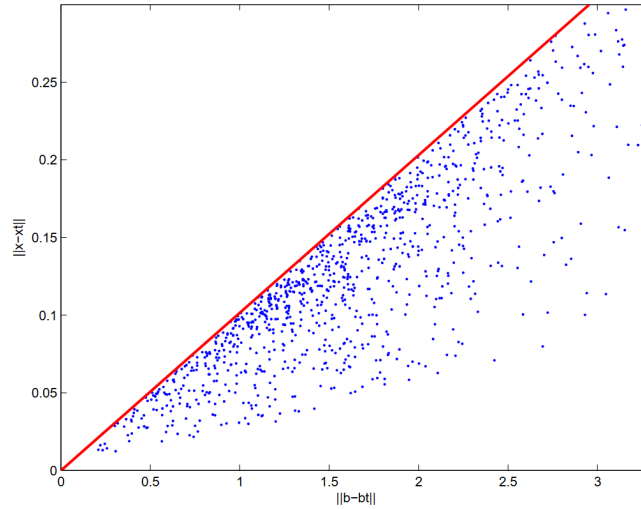


Figure 11.3: Upper bound of error amplification via $\|A^{-1}\|_2$

11.5.2 Relative Error and Conditioning

From $\|Ax\|_2 \leq \|A\|_2 \|x\|_2$ and the above:

$$\frac{\|\delta x\|_2}{\|x\|_2} \leq \|A\|_2 \|A^{-1}\|_2 \frac{\|\delta b\|_2}{\|b\|_2}.$$

[Condition Number]

$$\kappa(A) = \|A\|_2 \|A^{-1}\|_2.$$

Large $\kappa(A)$ implies the system is ill-conditioned.

11.5.3 Practical Meaning of the Condition Number

The condition number $\kappa(A)$ measures the worst-case amplification of relative errors:

- $\kappa(A) \approx 1$: well-conditioned system (stable).
- $\kappa(A) \gg 1$: ill-conditioned system (sensitive).

A large condition number indicates that the problem itself is difficult, not that the numerical method is poor.

```
A = np.array([[3,12,10],[12,0,20],[0,2,30]])
```

```
b1 = np.array([72.3, 99.5, 56.6])
```

```
x1 = np.linalg.solve(A,b1) #correct solution
```

```
b2 = np.array([73.3, 98.4, 57.1])
```

```

x2 = np.linalg.solve(A,b2) #perturbed solution

err_b = np.abs(b2 - b1)
rel_err_b = np.linalg.norm(err_b)/np.linalg.norm(b1)

err_x = np.abs(x2 - x1)
rel_err_x = np.linalg.norm(err_x)/np.linalg.norm(x1)
print("Relative error in b:", rel_err_b)
print("Relative error in x:", rel_err_x)

err_x_bound = np.linalg.cond(A) * rel_err_b
print("Error bound on relative error in x:", err_x_bound)

```

11.6 Residual and Backward Error

For an approximate solution $\tilde{x}$, the residual is

$$r = b - A\tilde{x}.$$

It satisfies:

$$\frac{\|\tilde{x} - x\|_2}{\|x\|_2} \leq \kappa(A) \frac{\|r\|_2}{\|b\|_2}.$$

A small residual does not necessarily imply a small error when $\kappa(A)$ is large.

11.6.1 Forward vs Backward Error

- **Forward error** measures the distance between the exact and computed solution.
- **Backward error** measures how much the original problem must be perturbed for the computed solution to be exact.

A numerically stable algorithm produces a small backward error, even when the forward error is large due to ill-conditioning.